

AI USAGE
VARDAAN GOEL
2025117001

write a Makefile using these flags:
gcc -std=c23 \
-D_POSIX_C_SOURCE=200809L \
-D_XOPEN_SOURCE=700 \
-Wall -Wextra -Werror \
-Wno-unused-parameter \
-fno-asm \
your_file.c

Makefile



```
CC = gcc
CFLAGS = -std=c23 -D_POSIX_C_SOURCE=200809L -D_XOPEN_SOURCE=700 -Wall -Wextra -Werror:

SRC_DIR = src
SRCS = $(wildcard $(SRC_DIR)/*.c)
OBJS = $(SRCS:.c=.o)
TARGET = shell.out

all: $(TARGET)

$(TARGET): $(OBJS)
    $(CC) $(CFLAGS) -o $@ $^

$(SRC_DIR)/%.o: $(SRC_DIR)/%.c
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    rm -f $(SRC_DIR)/*.o $(TARGET)
```

+ Ask Gemini

Pro Extended



read file line by line parse num, time and path. if path contains the dest substring find score for it score should be from 1 to 0.1 which goes down linearly and becomes 0.1 after 90 days

Here is the exact C code block generated from your prompt:

C



```
while (fgets(line, sizeof(line), f)) {
    HopEntry entry;
    if (sscanf(line, "%d %ld %[^\\n]", &entry.freq, &entry.timestamp, entry.path)

        if (strstr(entry.path, dest) != NULL) {
            double age_seconds = difftime(current_time, entry.timestamp);
            double max_age = 90.0 * 24.0 * 3600.0; // 90 days

            double factor = 1.0 - (age_seconds / max_age);
            if (factor < 0.1) {
                factor = 0.1;
            }

            entry.score = entry.freq * factor;
            matches[match_count++] = entry;
        }
    }
}
fclose(f);
```

+ Ask Gemini

Pro Extended



Gemini is AI and can make mistakes.

whats the way to get current user's name, hostname, and current working directory so i can build a prompt. include handling of errors in case anything fails



To get those details in C, you'll need to use a few standard POSIX system calls. Here is the code block that safely fetches the username, hostname, and working directory, complete with fallback values if any of the system calls fail.

You'll need these headers included first:

C



```
#include <unistd.h>
#include <pwd.h>
#include <limits.h>
#include <string.h>
#include <stdio.h>
```

Here is the logic to populate your variables:

C



```
char host[HOST_NAME_MAX];
char cwd[PATH_MAX];
char *username = NULL;

// 1. Get the Username
struct passwd *pw = getpwuid(getuid());
if (pw) {
```

+ Ask Gemini

Pro Extended ▼



Gemini is AI and can make mistakes.

```

struct passwd *pw = getpwuid(getuid());
if (pw) {
    username = pw->pw_name;
}
if (username == NULL) {
    username = "unknown"; // Fallback if user is not found
}

// 2. Get the Hostname
if (gethostname(host, sizeof(host)) != 0) {
    strncpy(host, "unknown", sizeof(host)); // Fallback if hostname fails
}

// 3. Get the Current Working Directory
if (getcwd(cwd, sizeof(cwd)) == NULL) {
    perror("getcwd failed"); // Print the specific error to stderr
    return; // Exit the prompt function since we can't display the path
}

```

How it works:

- `getuid()` + `getpwuid()` : `getuid` fetches the user ID of the calling process, and `getpwuid` looks up the corresponding password database entry to grab the human-readable username.
- `gethostname()` : Writes the system's hostname into your provided buffer. If it returns anything other than `0`, it failed, so we manually copy "unknown" into the buffer.
- `getcwd()` : Grabs the absolute path of the current directory. If it returns `NULL` (usually due to permissions or the directory being deleted out from under you), it triggers `perror` to print the exact system error.

👍 🗨 📄 ...

+ Ask Gemini

Pro Extended ▾



Gemini is AI and can make mistakes.

write function to get file path for shell's history file and write to given buffer, place file in home directory only failure is home directory being null

Here is a C function that retrieves the home directory using `getenv("HOME")` and constructs the full path to a shell history file (e.g., `~/.shell_history`).

As requested, the only point of failure it checks for is if the home directory environment variable returns `NULL`.

C



```
#include <stdio.h>
#include <stdlib.h>

/**
 * Gets the shell history file path and writes it to the provided buffer.
 *
 * @param buffer      The character array where the path will be written.
 * @param buffer_size The maximum size of the buffer to prevent overflows.
 * @return            0 on success, -1 if the home directory is NULL.
 */
int get_history_path(char *buffer, size_t buffer_size) {
    const char *home_dir = getenv("HOME");

    // The only failure condition: HOME directory is null
    if (home_dir == NULL) {
        return -1;
    }

    // Write the path to the buffer (change ".shell_history" to your specific file name)
    snprintf(buffer, buffer_size, "%s/.shell_history", home_dir);

    return 0;
}
```

+ Ask Gemini

Pro v



Gemini is AI and can make mistakes.

in c function, get the metadata for a given path, if it is a file print its info if it is directory, open it and print every file and directory in it, check for l flag and do accordingly

Here is a complete C program that implements this behavior. It uses standard POSIX system calls to check file metadata (`stat` / `lstat`) and traverse directories (`opendir` / `readdir`).

It mimics the behavior of the `ls` command, checking a boolean `l_flag` to decide whether to print just the names or the detailed metadata (permissions, owner, size, and modification time).

The C Implementation

C



```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>
#include <dirent.h>
#include <pwd.h>
#include <grp.h>
#include <time.h>
#include <string.h>
#include <stdbool.h>

// Helper function to print file permissions in the "-rw-r--r--" format
void print_permissions(mode_t mode) {
    printf((S_ISDIR(mode)) ? "d" : (S_ISLNK(mode)) ? "l" : "-");
    printf((mode & S_IRUSR) ? "r" : "-");
    printf((mode & S_IWUSR) ? "w" : "-");
    printf((mode & S_IXUSR) ? "x" : "-");
    printf((mode & S_IRGRP) ? "r" : "-");
```

+ Ask Gemini

Pro ▾



Gemini is AI and can make mistakes.

```

        printf((mode & S_IXUSR) ? "x" : "-");
        printf((mode & S_IRGRP) ? "r" : "-");
        printf((mode & S_IWGRP) ? "w" : "-");
        printf((mode & S_IXGRP) ? "x" : "-");
        printf((mode & S_IROTH) ? "r" : "-");
        printf((mode & S_IWOTH) ? "w" : "-");
        printf((mode & S_IXOTH) ? "x" : "-");
    }

    // Prints the information for a single file or directory entry
    void print_file_info(const char *name, const struct stat *sb, bool l_flag) {
        if (!l_flag) {
            printf("%s\n", name);
            return;
        }

        // Long format (-l) requires gathering user, group, and time strings
        print_permissions(sb->st_mode);

        struct passwd *pw = getpwuid(sb->st_uid);
        struct group *gr = getgrgid(sb->st_gid);

        char time_buf[256];
        strftime(time_buf, sizeof(time_buf), "%b %d %H:%M", localtime(&sb->st_mtime));

        printf(" %2lu %-8s %-8s %8ld %s %s\n",
            (unsigned long)sb->st_nlink,
            pw ? pw->pw_name : "unknown",
            gr ? gr->gr_name : "unknown",
            (long)sb->st_size,
            time_buf,
            name);
    }

    // Core function to process the given path
    void process_path(const char *path, bool l_flag) {
        struct stat sb;

```

+ Ask Gemini

Pro



Gemini is AI and can make mistakes.


```

C
void process_path(const char *path, bool l_flag) {
    struct stat sb;

    // Get metadata for the target path (lstat doesn't follow the symlink itself)
    if (lstat(path, &sb) == -1) {
        perror("Error reading path");
        return;
    }

    // Check if it is a directory
    if (S_ISDIR(sb.st_mode)) {
        DIR *dir = opendir(path);
        if (!dir) {
            perror("Error opening directory");
            return;
        }

        struct dirent *entry;
        while ((entry = readdir(dir)) != NULL) {
            // Skip the current (.) and parent (..) directory pointers
            if (strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name, "..") == 0)
                continue;
        }

        // Construct the full path so stat() can find the file
        char full_path[1024];
        snprintf(full_path, sizeof(full_path), "%s/%s", path, entry->d_name);

        struct stat entry_stat;
        if (lstat(full_path, &entry_stat) == -1) {
            continue; // Skip if we can't read the metadata
        }

        print_file_info(entry->d_name, &entry_stat, l_flag);
    }
    closedir(dir);
} else {
    // It's a standard file or symlink
    // ... (code for file processing) ...
}

```

+ Ask Gemini

Pro



Gemini is AI and can make mistakes.

write the function to print file's lines in reverse order using lseek, reading file in small chunks

Here is the complete `print_rev_file` function.

Reading a file backwards in chunks is tricky because lines can be split across two different chunks. To solve this, the algorithm uses a dynamically allocated `tail` buffer to carry incomplete lines from the end of one chunk into the beginning of the next.

C



```
static void print_rev_file(FileInfo *f, int n_flag) {
    int fd = open(f->name, O_RDONLY);
    if (fd == -1) {
        perror("peek");
        return;
    }

    off_t pos = lseek(fd, 0, SEEK_END);
    char chunk[4096];

    // 'tail' stores the piece of a line that got cut off by the chunk boundary
    char *tail = NULL;
    size_t tail_size = 0;

    int is_first = 1;
    int current_num = f->start_num;

    // Loop backwards until we hit the start of the file (pos == 0)
```

+ Ask Gemini

Pro Extended



Gemini is AI and can make mistakes.

```

// Loop backwards until we hit the start of the file (pos == 0)
while (pos > 0) {
    // 1. Calculate how much to read (a full 4096 bytes, or whatever is left)
    off_t to_read = (pos > (off_t)sizeof(chunk)) ? (off_t)sizeof(chunk) : pos;
    pos -= to_read;

    // 2. Move the read head and grab the chunk
    lseek(fd, pos, SEEK_SET);
    read(fd, chunk, to_read);

    int end = to_read - 1;

    // 3. Ignore the very last newline in the file so we don't print a phantom b'
    if (is_first && chunk[end] == '\n') {
        end--;
    }
    is_first = 0;

    // 4. Scan backwards through the chunk looking for newlines
    for (int j = end; j >= 0; j--) {
        if (chunk[j] == '\n') {
            size_t p_len = end - j;
            const char *part = chunk + j + 1;

            // Check if the reconstructed line will be completely empty
            int empty = is_empty(part, p_len) && (tail_size == 0 || is_empty(tail));

            if (n_flag && !empty) {
                printf("%d ", current_num--);
            }

            // Print the text from this chunk, followed immediately by the leftover
            if (p_len > 0) fwrite(part, 1, p_len, stdout);
        }
    }
}

```

+ Ask Gemini

Pro Extended



Gemini is AI and can make mistakes.

```

        // Print the text from this chunk, followed immediately by the leftover
        if (p_len > 0) fwrite(part, 1, p_len, stdout);
        if (tail_size > 0) fwrite(tail, 1, tail_size, stdout);
        printf("\n");

        // We finished printing this line, so wipe the tail clean
        free(tail);
        tail = NULL;
        tail_size = 0;

        end = j - 1; // Move the end marker to just before this newline
    }
}

// 5. If we hit the start of the chunk but haven't found a newline yet,
// we save the remaining text into the tail for the next loop iteration.
if (end >= 0) {
    size_t p_len = end + 1;
    char *new_tail = malloc(p_len + tail_size);

    // Note: because we are reading backwards, the new chunk data comes BEFORE
    memcpy(new_tail, chunk, p_len);
    if (tail_size > 0) {
        memcpy(new_tail + p_len, tail, tail_size);
    }

    free(tail);
    tail = new_tail;
    tail_size += p_len;
}
}

```

// 6. After the loop, if there is anything left in the tail, add it to the new chunk

+ Ask Gemini

Pro Extended



Gemini is AI and can make mistakes.

```

        if (end >= 0) {
            size_t p_len = end + 1;
            char *new_tail = malloc(p_len + tail_size);

            // Note: because we are reading backwards, the new chunk data comes BEFORE
            memcpy(new_tail, chunk, p_len);
            if (tail_size > 0) {
                memcpy(new_tail + p_len, tail, tail_size);
            }

            free(tail);
            tail = new_tail;
            tail_size += p_len;
        }
    }

    // 6. After the loop, if there is anything left in the tail, it's the very first
    if (tail_size > 0) {
        if (n_flag && !is_empty(tail, tail_size)) {
            printf("%d ", current_num--);
        }
        fwrite(tail, 1, tail_size, stdout);
        printf("\n");
        free(tail);
    }

    close(fd);
}

```

👍 💬 ↻ 📄 ...

+ Ask Gemini

Pro Extended ▾



Gemini is AI and can make mistakes.

function to read all lines from stdin into memory and store them array of strings strip all\n and \r

Here is the complete `buffer_stdin` function that uses a dynamically resizing array to safely load all of standard input into memory, while stripping out the newline and carriage return characters:

C



```
static void buffer_stdin(FileInfo *f) {
    // Start with a reasonable capacity and allocate memory
    int cap = 128;
    f->lines = malloc(cap * sizeof(char*));
    f->total_lines = 0;
    f->valid_lines = 0;

    char *line = NULL;
    size_t len = 0;
    ssize_t read_bytes;

    // Read line by line until EOF (Ctrl+D)
    while ((read_bytes = getline(&line, &len, stdin)) != -1) {
        // If our array is full, double its capacity
        if (f->total_lines >= cap) {
            cap *= 2;
            f->lines = realloc(f->lines, cap * sizeof(char*));
        }

        // Strip trailing \n (newline)
        if (read_bytes > 0 && line[read_bytes - 1] == '\n') {
            line[read_bytes - 1] = '\0';
        }
    }
}
```

+ Ask Gemini

Pro Extended



Gemini is AI and can make mistakes.

```

    f->total_lines = 0;
    f->valid_lines = 0;

    char *line = NULL;
    size_t len = 0;
    ssize_t read_bytes;

    // Read line by line until EOF (Ctrl+D)
    while ((read_bytes = getline(&line, &len, stdin)) != -1) {
        // If our array is full, double its capacity
        if (f->total_lines >= cap) {
            cap *= 2;
            f->lines = realloc(f->lines, cap * sizeof(char*));
        }

        // Strip trailing \n (newline)
        if (read_bytes > 0 && line[read_bytes - 1] == '\n') {
            line[read_bytes - 1] = '\0';
        }

        // Strip trailing \r (carriage return, common in Windows files)
        if (read_bytes > 1 && line[read_bytes - 2] == '\r') {
            line[read_bytes - 2] = '\0';
        }

        // Duplicate the cleaned string and store it in our array
        f->lines[f->total_lines++] = strdup(line);

        // Check if the line has visible text for line numbering (-n)
        if (!is_empty(line, strlen(line))) {
            f->valid_lines++;
        }
    }

    // getline allocates memory for the buffer, so we must free it
    free(line);
}

```

Answer now

+ Ask Gemini

Pro Extended ▾



Gemini is AI and can make mistakes.

c function that returns 1 if path is valid file and it is executable and 0 otherwise

Here is the C function to do this. It uses `stat()` to ensure the path is a regular file (ignoring directories) and `access()` to verify the current user has execute permissions.

C



```
#include <sys/stat.h>
#include <unistd.h>

int is_executable_file(const char *path) {
    struct stat st;

    // stat() retrieves file information and follows symlinks. Returns 0 on success.
    if (stat(path, &st) == 0) {
        // S_ISREG checks if it's a regular file (not a directory or device)
        // access(..., X_OK) checks if the current user has execute permission
        if (S_ISREG(st.st_mode) && access(path, X_OK) == 0) {
            return 1;
        }
    }

    return 0; // Not a file, not executable, or doesn't exist
}
```

+ Ask Gemini

Pro v



Gemini is AI and can make mistakes.


```

C
int is_executable_file(const char *path) {
    struct stat st;

    // stat() retrieves file information and follows symlinks. Returns 0 on success
    if (stat(path, &st) == 0) {
        // S_ISREG checks if it's a regular file (not a directory or device)
        // access(..., X_OK) checks if the current user has execute permission
        if (S_ISREG(st.st_mode) && access(path, X_OK) == 0) {
            return 1;
        }
    }

    return 0; // Not a file, not executable, or doesn't exist
}

```

How it works:

1. `stat(path, &st)` : Fetches the file's metadata. If it returns `-1`, the file doesn't exist or cannot be accessed, so the function safely returns `0`.
2. `S_ISREG(st.st_mode)` : Ensures the path points to a standard file. (Without this, the function might return `1` for a directory, as directories also have "execute" bits to allow entering them).
3. `access(path, X_OK)` : Specifically asks the operating system if the user running the program is allowed to execute the target path.

👍 🗨️ ↻ 📄 ...

+ Ask Gemini

Pro ▾



Gemini is AI and can make mistakes.

write in c a block using fork() to create child processes.
execute programs in the child processes

Here is the standard C code block for forking a process and executing a program.

This uses the classic `fork()` + `execv()` + `waitpid()` pattern, which is the foundational way UNIX systems (and custom shells) run commands!

C



```
// Create a new child process
pid_t pid = fork();

if (pid < 0) {
    // Fork failed (e.g., out of memory or process limits reached)
    perror("fork error");
}
else if (pid == 0) {
    // --- CHILD PROCESS ---
    // execv completely replaces this child process with the target program.
    // It requires the absolute/resolved path, and a NULL-terminated array of arguments.
    execv(resolved_path, exec_args);

    // IMPORTANT: execv NEVER returns if it succeeds.
    // If the code reaches this line, it means the execution failed!
    perror("execv error");
    // (never returns)
}
```

+ Ask Gemini

Pro Extended ▾



Gemini is AI and can make mistakes.

```

}
else if (pid == 0) {
    // --- CHILD PROCESS ---
    // execv completely replaces this child process with the target program.
    // It requires the absolute/resolved path, and a NULL-terminated array of arguments.
    execv(resolved_path, exec_args);

    // IMPORTANT: execv NEVER returns if it succeeds.
    // If the code reaches this line, it means the execution failed!
    perror("execv error");
    exit(EXIT_FAILURE);
}
else {
    // --- PARENT PROCESS ---
    // The parent (your shell) waits here until the child process terminates.
    int status;
    waitpid(pid, &status, 0);
}

```

How this works:

1. `fork()` clones your shell. For a split second, there are two identical versions of your shell running at the exact same time.
2. In the child clone (`pid == 0`), `execv()` overwrites the clone's memory with the new program (like `ls` or `echo`) and runs it.
3. In the parent (`pid > 0`), `waitpid()` pauses your shell so it doesn't print the next

+ Ask Gemini

Pro Extended ▼



Gemini is AI and can make mistakes.

c block that sets up feeder for multiple input files, for more than 1 files create pipe and fork a new process inside the child read file descriptors data into a buffer and write them to end of pipe

Here is the C code block that sets up a feeder process to stream data from multiple input files into a pipe:

C



```
int pipefd[2] = {-1, -1};
pid_t feeder_pid = -1;

// If there are multiple input files, we need a feeder process to combine them
if (in_count > 1) {
    if (pipe(pipefd) < 0) {
        perror("pipe error");
    }

    feeder_pid = fork();
    if (feeder_pid == 0) {
        // --- FEEDER CHILD PROCESS ---
        // Close the read end of the pipe since this process only writes to it
        close(pipefd[0]);

        char buf[4096];
```

+ Ask Gemini

Flash-Lite ▾



Gemini is AI and can make mistakes.

```

    if (pipe(pipefd) < 0) {
        perror("pipe error");
    }

    feeder_pid = fork();
    if (feeder_pid == 0) {
        // --- FEEDER CHILD PROCESS ---
        // Close the read end of the pipe since this process only writes to it
        close(pipefd[0]);

        char buf[4096];
        ssize_t bytes_read;

        // Loop through each input file descriptor sequentially
        for (int i = 0; i < in_count; i++) {
            // Read from the file and write its contents into the pipe's write-end
            while ((bytes_read = read(in_fds[i], buf, sizeof(buf))) > 0) {
                write(pipefd[1], buf, bytes_read);
            }
            // Close each file descriptor as we finish reading it
            close(in_fds[i]);
        }

        // Close the write end of the pipe to signal EOF to the main process, then
        close(pipefd[1]);
        exit(EXIT_SUCCESS);
    }
}

```

+ Ask Gemini

Flash-Lite ▾



Gemini is AI and can make mistakes.

c block to fork child process to run command with input redirection. if 1 file, use dup2 to redirect to STDIN_FILENO if more than 1 redirect to stdin then call execl to run command.

Here is the C code block for forking the process and applying the input redirection:

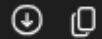
C



```
pid_t pid = fork();
if (pid < 0) {
    perror("fork error");
}
else if (pid == 0) {
    // --- CHILD PROCESS ---
    if (in_count == 1) {
        // Single file: use dup2 to redirect the file descriptor to STDIN
        dup2(in_fds[0], STDIN_FILENO);
    } else if (in_count > 1) {
        // Multiple files: redirect the read-end of the pipe to STDIN
        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[0]);
        close(pipefd[1]);
    }

    // Close all raw file descriptors in the child
```

C



```
        dup2(in_fds[0], STDIN_FILENO);
    } else if (in_count > 1) {
        // Multiple files: redirect the read-end of the pipe to STDIN
        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[0]);
        close(pipefd[1]);
    }

    // Close all raw file descriptors in the child
    for (int i = 0; i < in_count; i++) close(in_fds[i]);

    // Run the command
    execv(p, args);
    perror("execv error");
    exit(EXIT_FAILURE);
}
else {
    // --- PARENT PROCESS ---
    if (in_count > 1) {
        close(pipefd[0]);
        close(pipefd[1]);
    }

    for (int i = 0; i < in_count; i++) close(in_fds[i]);

    // Wait for child processes to finish
    waitpid(pid, NULL, 0);
    if (feeder_pid > 0) {
        waitpid(feeder_pid, NULL, 0);
    }
}
```

c block to open multiple files for output redirection. check file flags and open file with 0644 permissions

Here is the C code block to handle opening multiple output files, setting the correct flags, applying 0644 permissions, and handling errors cleanly:

C



```
int out_fds[1000];

for (int i = 0; i < out_count; i++) {
    // Determine flags: always Write-Only and Create, then choose Append or Trunc
    int flags = O_WRONLY | O_CREAT | (out_append[i] ? O_APPEND : O_TRUNC);

    // Open with 0644 permissions (rw-r--r--)
    out_fds[i] = open(out_files[i], flags, 0644);

    // If the file fails to open (e.g., permission denied or invalid path)
    if (out_fds[i] < 0) {
        fprintf(stderr, "cshell: unable to create file for writing\n");

        // Cleanup: Close all previously opened input files
        for (int j = 0; j < in_count; j++) close(in_fds[j]);

        // Cleanup: Close all successfully opened output files before this failure
        for (int j = 0; j < i; j++) close(out_fds[j]);
    }
}
```

+ Ask Gemini

Pro Extended ▾



Gemini is AI and can make mistakes.



```
int out_fds[1000];

for (int i = 0; i < out_count; i++) {
    // Determine flags: always Write-Only and Create, then choose Append or Trunc
    int flags = O_WRONLY | O_CREAT | (out_append[i] ? O_APPEND : O_TRUNC);

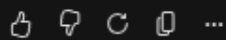
    // Open with 0644 permissions (rw-r--r--)
    out_fds[i] = open(out_files[i], flags, 0644);

    // If the file fails to open (e.g., permission denied or invalid path)
    if (out_fds[i] < 0) {
        fprintf(stderr, "cshell: unable to create file for writing\n");

        // Cleanup: Close all previously opened input files
        for (int j = 0; j < in_count; j++) close(in_fds[j]);

        // Cleanup: Close all successfully opened output files before this failure
        for (int j = 0; j < i; j++) close(out_fds[j]);

        return; // Abort execution completely
    }
}
```



+ Ask Gemini

Pro Extended ▾



Gemini is AI and can make mistakes.

c block to set up a loop for executing pipeline of commands. store the process ids for them and iterate through them get pointer to commands arguments handle errors

Here is the C code block to initialize the pipeline tracking variables and set up the main execution loop, exactly as requested:

C



```
int in_fd = STDIN_FILENO; // Tracks the read-end of the previous pipe
pid_t pids[256];
pid_t feeder_pids[256];
pid_t writer_pids[256];

// --- 2. LOOP THROUGH EACH COMMAND IN THE PIPELINE ---
for (int c = 0; c < num_cmds; c++) {
    int start = pipe_idx[c] + 1;
    int end = pipe_idx[c + 1];
    int cmd_count = end - start;
    char** cmd_input = &input[start];

    // Check for empty commands (e.g., consecutive pipes like `||` or dangling `
    if (cmd_count == 0) {
        fprintf(stderr, "cshell: syntax error near unexpected token `|\n");
        break;
    }
}
```

+ Ask Gemini

Pro Extended ▾



Gemini is AI and can make mistakes.

```

int in_fd = STDIN_FILENO; // Tracks the read-end of the previous pipe
pid_t pids[256];
pid_t feeder_pids[256];
pid_t writer_pids[256];

// --- 2. LOOP THROUGH EACH COMMAND IN THE PIPELINE ---
for (int c = 0; c < num_cmds; c++) {
    int start = pipe_idx[c] + 1;
    int end = pipe_idx[c + 1];
    int cmd_count = end - start;
    char** cmd_input = &input[start];

    // Check for empty commands (e.g., consecutive pipes like `||` or dangling `
    if (cmd_count == 0) {
        fprintf(stderr, "cshell: syntax error near unexpected token `|\n");
        break;
    }

    // Setup the pipe for the NEXT command to read from (if we aren't the last co
    int fd[2] = {-1, -1};
    if (c < num_cmds - 1) {
        if (pipe(fd) < 0) {
            perror("pipe error");
        }
    }
}

```

👍 🗨️ ↻ 📄 ...

+ Ask Gemini

Pro Extended ▼



Gemini is AI and can make mistakes.

write C code for SIGCHLD signal handler for c shell, handle all failures, use waipid() with WNOHANG, use WIFEXITED, WIFSIGNALED if needed, explain what you are doing and how its done



C



```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>
#include <errno.h>
#include <string.h>

#define MAX_JOBS 1024

// Process tracking structure for background jobs
typedef struct {
    pid_t pid;
    int job_id;
    char name[256];
    int active;           // 1 if currently running
    int completed;       // 1 if reaped by signal handler, ready to report
    int normal_exit;     // 1 if exited via exit(), 0 if killed by signal
} job_t;

job_t jobs[MAX_JOBS];
```

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes

```

C
    int exit_code;    // Exit status code or terminating signal number
} ProcessJob;

static ProcessJob bg_jobs[MAX_JOBS];
static int job_count = 0;

/*
 * SIGCHLD Signal Handler
 * Reaps background processes safely without blocking the main process thread.
 */
void sigchld_handler(int sig) {
    // 1. Preserve original errno to protect main-thread system calls
    int saved_errno = errno;

    int status;
    pid_t pid;

    // 2. Non-blocking loop to reap ALL finished child processes
    while ((pid = waitpid(-1, &status, WNOHANG)) > 0) {
        for (int i = 0; i < job_count; i++) {
            if (bg_jobs[i].active && bg_jobs[i].pid == pid) {
                bg_jobs[i].completed = 1;

                // 3. Inspect child termination reason
                if (WIFEXITED(status)) {
                    bg_jobs[i].normal_exit = 1;
                    bg_jobs[i].exit_code = WEXITSTATUS(status);
                } else if (WIFSIGNALED(status)) {
                    bg_jobs[i].normal_exit = 0;
                    bg_jobs[i].exit_code = WTERMSIG(status);
                }
            }
        }
    }
}

```

+ Ask Gemini

Flash Extended



```

        }
    }

    // 4. Restore original errno state
    errno = saved_errno;
}

/*
 * Registers the SIGCHLD handler using sigaction
 */
void setup_sigchld_handler(void) {
    struct sigaction sa;
    sa.sa_handler = sigchld_handler;
    sigemptyset(&sa.sa_mask);

    // SA_RESTART prevents system calls like getline() from failing with EINTR
    // SA_NOCLDSTOP ignores signals from stopped/continued processes (processes paused)
    sa.sa_flags = SA_RESTART | SA_NOCLDSTOP;

    if (sigaction(SIGCHLD, &sa, NULL) < 0) {
        perror("cshell: sigaction registration failed");
        exit(EXIT_FAILURE);
    }
}

/*
 * Safe print function called directly from the main loop before printing prompt
 */
void check_and_print_bg_jobs(void) {

```

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.

```

void check_and_print_bg_jobs(void) {
    for (int i = 0; i < job_count; i++) {
        if (bg_jobs[i].active && bg_jobs[i].completed) {
            if (bg_jobs[i].normal_exit) {
                printf("%s with pid %d exited normally\n", bg_jobs[i].name, (int)bg_jobs[i].pid);
            } else {
                printf("%s with pid %d exited abnormally\n", bg_jobs[i].name, (int)bg_jobs[i].pid);
            }
            fflush(stdout);
            bg_jobs[i].active = 0; // Mark slot as free for reuse
        }
    }
}

```

Technical Breakdown

- `while ((pid = waitpid(-1, &status, WNOHANG)) > 0)`
 - **Why `-1`:** Instructs `waitpid` to wait for *any* child process belonging to the parent shell.
 - **Why `WNOHANG`:** Prevents the signal handler from blocking/freezing if no child processes have finished.
 - **Why a `while` loop:** UNIX signals do not queue. If three background jobs finish at the exact same millisecond, the operating system kernel triggers only **one** `SIGCHLD` signal. The `while` loop continuously collects all pending zombie processes until `waitpid` returns `0` (no more finished children) or `-1` (error/no children left).

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.

- **Preserving `errno`** (`int saved_errno = errno; ... errno = saved_errno;`)
 - Signals interrupt the main execution thread asynchronously. If the main shell thread was in the middle of executing a system call (like `open()` or `read()`) that sets `errno` , the signal handler's call to `waitpid()` will overwrite `errno` with `ECHILD` when the loop ends.
 - Saving `errno` at the start of the handler and restoring it at the end prevents corrupted error codes in the main shell loop.
- **Status Extraction Macros (`WIFEXITED` , `WIFSIGNALED`)**
 - `WIFEXITED(status)` : Returns true if the child exited through standard execution termination (calling `exit()` or returning from `main()`).
 - `WEXITSTATUS(status)` : Extracts the integer exit code (0–255) returned by the child process.
 - `WIFSIGNALED(status)` : Returns true if the process was abnormally killed by an unhandled signal (e.g., `SIGKILL` , `SIGSEGV`).
 - `WTERMSIG(status)` : Extracts the signal number that caused process termination.
- **Async-Signal Safety (No `printf` in Handler)**
 - Standard I/O functions like `printf()` or memory management like `malloc()` are **non-reentrant** and not safe to call inside signal handlers. Calling `printf()` in a signal handler can trigger deadlocks or output buffer corruption if interrupted while the main thread is also writing to standard output.
 - The handler updates shared flag variables (`completed = 1`), leaving `check_and_print_bg_jobs()` to safely execute `printf()` from the main

+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

- **Status Extraction Macros (`WIFEXITED` , `WIFSIGNALED`)**
 - `WIFEXITED(status)` : Returns true if the child exited through standard execution termination (calling `exit()` or returning from `main()`).
 - `WEXITSTATUS(status)` : Extracts the integer exit code (0–255) returned by the child process.
 - `WIFSIGNALED(status)` : Returns true if the process was abnormally killed by an unhandled signal (e.g., `SIGKILL` , `SIGSEGV`).
 - `WTERMSIG(status)` : Extracts the signal number that caused process termination.
- **Async-Signal Safety (No `printf` in Handler)**
 - Standard I/O functions like `printf()` or memory management like `malloc()` are **non-reentrant** and not safe to call inside signal handlers. Calling `printf()` in a signal handler can trigger deadlocks or output buffer corruption if interrupted while the main thread is also writing to standard output.
 - The handler updates shared flag variables (`completed = 1`), leaving `check_and_print_bg_jobs()` to safely execute `printf()` from the main execution thread.
- **`sigaction` and `SA_RESTART`**
 - Using `sigaction` with `SA_RESTART` tells Linux to automatically restart interrupted blocking system calls (such as `getline()` waiting for keyboard input) instead of failing early with `EINTR` .

👍 🗨️ ↻ 📄 ...

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.

for c shell, i need it to support background processes using `&`, handle race conditions where i must print job id before eexecutin the child command i need a way to force child to wait until parent procoesses job id printing what should be

To synchronize the parent shell and child process without relying on arbitrary delays like `usleep()`, use an **unnamed synchronization pipe** (`pipe()`).

This mechanism creates a blocking execution barrier: the child process pauses at a `read()` call until the parent finishes printing the `[job_id] pid` header and closes its end of the pipe, which sends an End-of-File (EOF) to unblock the child immediately before `execvp()`.

How the Barrier Mechanism Works

1. **Pipe Creation:** Before calling `fork()`, the parent creates `pipe(sync_pipe)`.
2. **Child Barrier (`read()`):** The child closes `sync_pipe[1]` (write end) and executes `read(sync_pipe[0], &dummy, 1)`. Because no data is written to the pipe, `read()` blocks the child process completely.
3. **Parent Printing:** The parent prints `[job_id] pid` and calls `fflush(stdout)` to flush standard output.
4. **Child Release (`close()`):** The parent closes `sync_pipe[1]`. Closing the last write descriptor delivers an EOF signal to the pipe, causing the child's blocking `read()` to return `0` instantly so `execvp()` can run.

+ Ask Gemini

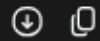
Flash Extended



Gemini is AI and can make mistakes.

C Implementation

c



```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <fcntl.h>
#include <errno.h>

void execute_cmd(char** args, int is_bg, int job_id) {
    int sync_pipe[2] = {-1, -1};

    // 1. Create synchronization pipe ONLY for background commands
    if (is_bg) {
        if (pipe(sync_pipe) < 0) {
            perror("cshell: pipe creation failed");
            return;
        }
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("cshell: fork failed");
        if (is_bg) {
            close(sync_pipe[0]);
        }
    }
```

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.

```

    if (!is_bg) {
        close(sync_pipe[0]);
        close(sync_pipe[1]);
    }
    return;
}

else if (pid == 0) {
    // --- CHILD PROCESS ---
    if (is_bg) {
        // Isolate background process into its own process group
        setpgid(0, 0);

        // Close unused write end in child
        close(sync_pipe[1]);

        // BLOCKING BARRIER: Pause child execution until parent prints job ID
        char dummy;
        read(sync_pipe[0], &dummy, 1); // Returns 0 (EOF) when parent closes sync
        close(sync_pipe[0]);

        // Prevent background process from stealing terminal input
        int dev_null = open("/dev/null", O_RDONLY);
        if (dev_null >= 0) {
            dup2(dev_null, STDIN_FILENO);
            close(dev_null);
        }
    }

    // Execute binary image
    execvp(args[0], args);
    perror("cshell: execution failed");
}

```

+ Ask Gemini

Flash Extended ▼



Gemini is AI and can make mistakes.

```

else {
    // --- PARENT PROCESS ---
    if (is_bg) {
        // Close unused read end in parent
        close(sync_pipe[0]);

        // 2. Print job header guaranteed BEFORE child execution output
        printf("[%d] %d\n", job_id, (int)pid);
        fflush(stdout);

        // 3. Unblock the child process by sending EOF through pipe closure
        close(sync_pipe[1]);
    } else {
        // Foreground command: parent waits for child process completion
        int status;
        waitpid(pid, &status, 0);
    }
}
}

```

Critical File Descriptor Rules

- **Always call** `fflush(stdout)` **immediately after** `printf("[%d] %d\n", ...)` **in the parent** to guarantee the job header is flushed to the terminal buffer before releasing the child.
- **Close unused ends early:** The child *must* close `sync_pipe[1]` before calling `read()`, otherwise, `read()` will block permanently waiting for data from itself.

+ Ask Gemini

Flash Extended ▾



```

        close(sync_pipe[1]);
    } else {
        // Foreground command: parent waits for child process completion
        int status;
        waitpid(pid, &status, 0);
    }
}
}
}

```

Critical File Descriptor Rules

- **Always call `fflush(stdout)`** immediately after `printf("[%d] %d\n", ...)` in the parent to guarantee the job header is flushed to the terminal buffer before releasing the child.
- **Close unused ends early:** The child *must* close `sync_pipe[1]` before calling `read()`, otherwise `read()` will block permanently waiting for data from itself.
- **Cleanup on Errors:** If `fork()` or path resolution fails before `fork()`, ensure both `sync_pipe[0]` and `sync_pipe[1]` are closed to prevent file descriptor leaks.

Want to test this pipe barrier with pipelines like `cat | wc -l &?`

→ Yes

👍 🗨️ ↻ 📄 ...

+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

for the c shell there is bug where completed foreground processes stay in my activities table and marked as running even after finishing,

```

int execute(char**input,int count,int bg){
if (input[0]==NULL||count==0)return 1;
int pipeindex[1000];
int n=1;
pipeindex[0]=-1;
for (int i=0;i<count;i++){
if (strcmp(input[i],"")==0){pipeindex[n++]=i;input[i]=NULL;}
}
pipeindex[n]=count;
int in_fd = STDIN_FILENO;
pid_t pids[256];
pid_t feeder_pids[256];
pid_t writer_pids[256];
char cmd_names[256][256];
for (int i=0;i<256;i++){
pids[i]=-1;
feeder_pids[i]=-1;
writer_pids[i]=-1;
}
int sync_pipe[2]={-1,-1};
if(bg&&pipe(sync_pipe)<0){perror("pipe error");}
for (int c = 0; c < n; c++) {
int start = pipeindex[c] + 1;
int end = pipeindex[c + 1];
int cmd_count = end - start;
char** cmd_input = &input[start];
if (cmd_count == 0) {
fprintf(stderr, "cshell: syntax error near unexpected token
'\n");
if (bg&&sync_pipe[0]!=-1)
{close(sync_pipe[0]);close(sync_pipe[1]);}
}
}

```

+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

```

}
int fd[2] = {-1, -1};
if (c < n - 1) {
if (pipe(fd) < 0) {
perror("pipe error");
if (bg&&sync_pipe[0]!=-1)
{close(sync_pipe[0]);close(sync_pipe[1]);}
return 0;
}
}
char*arg[1000];char*files[1000];int file_count=0;
char*out_files[1000];int out_count=0;
int out[1000];
int num=0;
for (int i=0;i<cmd_count;i++){
if (strcmp(cmd_input[i],"<")==0){
if (i+1>=cmd_count){fprintf(stderr,"cshell: syntax error\n");
if (bg&&sync_pipe[0]!=-1)
{close(sync_pipe[0]);close(sync_pipe[1]);}
return 0;}
else {files[file_count++]=cmd_input[i+1];i++;}
}
else if (strcmp(cmd_input[i],">")==0){
if (i+1>=cmd_count){fprintf(stderr,"cshell: syntax error\n"); if
(bg&&sync_pipe[0]!=-1)
{close(sync_pipe[0]);close(sync_pipe[1]);}
return 0;}
else
{out[out_count]=0;out_files[out_count]=cmd_input[i+1];out
_count++;i++;}
}
else if (strcmp(cmd_input[i],">>")==0){
if (i+1>=cmd_count){fprintf(stderr,"cshell: syntax error\n"); if
(bg&&sync_pipe[0]!=-1)

```

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.


```

return 0;
else
{out[out_count]=1;out_files[out_count]=cmd_input[i+1];out
_count++;i++;}

}
else {
arg[num++]=cmd_input[i];
}
}
if (num==0)continue;
arg[num]=NULL;
if (arg[0]!=NULL){if (arg[0][0]!='%')
strcpy(cmd_names[c],arg[0]);
else strcpy(cmd_names[c],arg[0]+1);
}
int stored[1000];
for (int i=0;i<file_count;i++){
stored[i]=open(files[i],O_RDONLY);
if (stored[i]<0){
fprintf(stderr,"cshell: no such file or directory\n");
for (int j=0;j<i;j++)close(stored[j]);
if (bg&&sync_pipe[0]!=-1
{close(sync_pipe[0]);close(sync_pipe[1]);}
return 0;}
}
int out_stored[1000];
for (int i=0;i<out_count;i++){
int flags = O_WRONLY | O_CREAT | (out[i] ? O_APPEND :
O_TRUNC);
out_stored[i] = open(out_files[i], flags, 0644);
if (out_stored[i] < 0) {
fprintf(stderr, "cshell: unable to create file for writing\n");
for (int j = 0; j < file_count; j++) close(stored[j]);
for (int i = 0; i < i; i++) close(out_stored[i]);
}
}
}

```

+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

```

    {close(sync_pipe[0]);close(sync_pipe[1]);}
    return 0;
}
}
char* cmd=arg[0];
char* p=path(cmd);
if (p==NULL){
if (cmd[0]=='%'){
fprintf(stderr,"cshell: command not found (%s)\n",cmd+1);

}
else {fprintf(stderr,"cshell: command not found
(%s)\n",cmd);}
for (int i=0;i<file_count;i++)close(stored[i]);
for (int i=0;i<out_count;i++)close(out_stored[i]);
if (bg&&sync_pipe[0]!=-1)
{close(sync_pipe[0]);close(sync_pipe[1]);}
return 0;
}
if (arg[0][0]=='%'){arg[0]++;}
int pipefd[2] = {-1, -1};
pid_t feeder_pid = -1;
if (file_count > 1) {
if (pipe(pipefd) < 0) {
perror("pipe error");
}
feeder_pid = fork();
if (feeder_pid == 0) {
close(pipefd[0]);
char buf[4096];
ssize_t bytes_read;
for (int i = 0; i < file_count; i++) {
while ((bytes_read = read(stored[i], buf, sizeof(buf))) > 0) {
write(pipefd[1], buf, bytes_read);

```

+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

```

write(pipefd[1], buf, bytes_read);
}
close(stored[i]);
}
close(pipefd[1]);
exit(EXIT_SUCCESS);
}
}

int out_pipefd[2]={-1,-1};
pid_t writer_pid=-1;
if (out_count>1){if (pipe(out_pipefd) < 0) {
perror("pipe error");
}
writer_pid=fork();
if (writer_pid==0){
close(out_pipefd[1]);
char buf[4096];
ssize_t bytes_read;
while ((bytes_read = read(out_pipefd[0], buf, sizeof(buf))) >
0) {
for (int i = 0; i < out_count; i++) {
write(out_stored[i], buf, bytes_read);
}}
close(out_pipefd[0]);
for (int i=0;i<out_count;i++)close(out_stored[i]);
exit(EXIT_SUCCESS);}
}

pid_t pid = fork();
if (pid < 0) {
perror("fork error");free(p);
if (bg&&sync_pipe[0]!=-1)
(close(sync_pipe[0]);close(sync_pipe[1]);)
}
}

```

+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

```

if (file_count == 1) {
    dup2(stored[0], STDIN_FILENO);
} else if (file_count > 1) {
    dup2(pipefd[0], STDIN_FILENO);
    close(pipefd[0]);
    close(pipefd[1]);
}
else if (in_fd != STDIN_FILENO) { dup2(in_fd, STDIN_FILENO); }
if (out_count == 1) { dup2(out_stored[0], STDOUT_FILENO); }
else if (out_count > 1) {
    dup2(out_pipefd[1], STDOUT_FILENO);
    close(out_pipefd[0]);
    close(out_pipefd[1]);
}
else if (c < n-1) {
    dup2(fd[1], STDOUT_FILENO);
}
if (in_fd != STDIN_FILENO) close (in_fd);
if (c < n-1) { close(fd[0]); close(fd[1]); }
for (int i = 0; i < file_count; i++) close(stored[i]);
for (int i = 0; i < out_count; i++) close(out_stored[i]);
execv(p, arg);
perror("execv error");
exit(EXIT_FAILURE);
}
else { pid_t ppid; pids[c] = pid;
if (c == 0) { ppid = pid; }
else ppid = pids[0];
setppid(pid, ppid);

if (file_count > 1) {
    close(pipefd[0]);
    close(pipefd[1]);
}
if (out_count > 1) {

```

+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

```

    }
    else{
        for (int c=0;c<n;c++){
            waitpid(pids[c], NULL, 0);
            if (feeder_pids[c] > 0) {
                waitpid(feeder_pids[c], NULL, 0);}
            if (writer_pids[c] > 0) {
                waitpid(writer_pids[c], NULL, 0);
            }
        }
        print_prs();
        return 1;
    }

    void update_state(){
        int status;
        pid_t pid;
        while((pid=waitpid(-1,&status,WNOHANG|WUNTRACED|WCONTINUED))>0){
            for (int i=0;i<job_count;i++){
                if (!jobs[i].active)continue;
                for (int j=0;j<jobs[i].numj++){
                    if (jobs[i].prs[j].pid==pid){
                        if (WIFEXITED(status))
                            {jobs[i].prs[j].state=2;jobs[i].prs[j].exit=1;}
                        else if (WIFSIGNALED(status))
                            {
                                jobs[i].prs[j].state=2;jobs[i].prs[j].exit=0;
                            }
                        else if (WIFSTOPPED(status)){jobs[i].prs[j].state=1;}
                        else if (WIFCONTINUED(status)){jobs[i].prs[j].state=0;}
                    }}
                int flag=0;
                for (int k=0;k<jobs[i].num;k++){
                    if (jobs[i].prs[k].state!=2){flag=1;break;}

```

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.

The root cause of this bug is how POSIX process reaping interacts with your job tracking table.

Root Cause Explanation

1. **State Consumption via `waitpid`:** In Linux, process exit status can only be read once. When `execute()` runs `waitpid(pids[c], NULL, 0)` for a foreground process, it reaps the process synchronously and clears it from the operating system process table.
2. **NULL Exit Status Discarded:** `execute()` passes `NULL` as the second argument to `waitpid()`, discarding the exit status entirely without updating the `jobs` array.
3. **`update_state()` Misses Reaped Processes:** When `update_state()` subsequently calls `waitpid(-1, &status, WNOHANG | ...)`, the kernel returns `0` or `-1` because the foreground child was already reaped. Because `update_state()` never sees the PID exit, the job remains marked as `state = 0` (Running) in your `jobs` table forever.
4. **Missing `WUNTRACED` Flag:** Using `0` instead of `WUNTRACED` in `execute()` causes `waitpid()` to block indefinitely if a foreground process is suspended using `Ctrl+Z` (`SIGTSTP`).

Solution

Refactor `update_state()` to accept optional explicit PID and status parameters (`update_state(pid_t target_pid, int target_status)`). When `execute()` reaps a foreground process, it captures `status` with `WUNTRACED` and passes it directly to `update_state()`.

Updated `activities.h`

c



```
#ifndef ACTIVITIES_H
#define ACTIVITIES_H

#include <sys/types.h>
```

+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

```

#ifndef ACTIVITIES_H
#define ACTIVITIES_H

#include <sys/types.h>

void sigchld(int sig);
void print_prs(void);
void print_activities(void);
void add_job(int job_id, pid_t pgid, int num, pid_t* pids, char cmd_names[][256], int*
void update_state(pid_t target_pid, int target_status);

#endif

```

Updated activities.c

c  

```

#define _DEFAULT_SOURCE
#include "activities.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <errno.h>

typedef struct {
    pid_t pid;
    char name[256];
    int state;
    int exit;
    int printed;
} process;

```

+ Ask Gemini

Flash Extended 



Gemini is AI and can make mistakes.

C



```
job jobs[2000];
int job_count = 0;

static void apply_proc_status(pid_t pid, int status) {
    for (int i = 0; i < job_count; i++) {
        if (!jobs[i].active) continue;
        for (int j = 0; j < jobs[i].num; j++) {
            if (jobs[i].prs[j].pid == pid) {
                if (WIFEXITED(status)) {
                    jobs[i].prs[j].state = 2;
                    jobs[i].prs[j].exit = 1;
                } else if (WIFSIGNALED(status)) {
                    jobs[i].prs[j].state = 2;
                    jobs[i].prs[j].exit = 0;
                } else if (WIFSTOPPED(status)) {
                    jobs[i].prs[j].state = 1;
                } else if (WIFCONTINUED(status)) {
                    jobs[i].prs[j].state = 0;
                }
            }
        }
        int flag = 0;
        for (int k = 0; k < jobs[i].num; k++) {
            if (jobs[i].prs[k].state != 2) { flag = 1; break; }
        }
        if (flag == 0) jobs[i].active = 0;
    }
}

void update_state(pid_t target_pid, int target_status) {
    if (target_pid > 0) {
        apply_proc_status(target_pid, target_status);
    } else {
        int status;
        pid_t pid;
        while ((pid = waitpid(-1, &status, WNOHANG | WUNTRACED | WCONTINUED)) > 0) {
            apply_proc_status(pid, status);
        }
    }
}
```

[+ Ask Gemini](#)

Flash Extended



Gemini is AI and can make mistakes.

Updated `execute()` Foreground Wait Block (`execution.c`)

Replace the foreground `else` block at the end of `execute()` with:

C



```
else {
    for (int c = 0; c < n; c++) {
        int status;
        if (pids[c] > 0) {
            waitpid(pids[c], &status, WUNTRACED);
            update_state(pids[c], status);
        }
        if (feeder_pids[c] > 0) {
            waitpid(feeder_pids[c], &status, 0);
        }
        if (writer_pids[c] > 0) {
            waitpid(writer_pids[c], &status, 0);
        }
    }
    print_prs();
}
return 1;
}
```

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.

for the c shell, i need to implement standard Ctrl+C, Ctrl+Z and Ctrl+D logic
use signal handler for Ctrl+C, ignore SIGTSTP, SIGTTIN, SIGTTOU use tcsetpgrp() to transfer terminal hold before execution and reclaim after
for ctrl+z use WUNTRACED,

Signal Handler Setup & Prompt Loop (`main.c`)

C

```
#define _DEFAULT_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>
#include <errno.h>
#include "activities.h"

void sigint_handler(int sig) {
    (void)sig;
    write(STDOUT_FILENO, "\n", 1);
}
```

+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

```

int main() {
    /* 1. SIGINT Handler: SA_RESTART is omitted so getline() breaks with EINTR on Ctrl-C */
    struct sigaction sa_int;
    sa_int.sa_handler = sigint_handler;
    sigemptyset(&sa_int.sa_mask);
    sa_int.sa_flags = 0;
    sigaction(SIGINT, &sa_int, NULL);

    /* 2. Ignore terminal/stop signals in parent shell */
    signal(SIGTSTP, SIG_IGN);
    signal(SIGTTIN, SIG_IGN);
    signal(SIGTTOU, SIG_IGN);

    /* 3. Register SIGCHLD handler for background process tracking */
    struct sigaction sa_chld;
    sa_chld.sa_handler = sigchld;
    sigemptyset(&sa_chld.sa_mask);
    sa_chld.sa_flags = SA_RESTART;
    sigaction(SIGCHLD, &sa_chld, NULL);

    /* Put parent shell into its own process group and claim terminal ownership */
    pid_t shell_pgid = getpid();
    setpgid(shell_pgid, shell_pgid);
    tcsetpgrp(STDIN_FILENO, shell_pgid);

    char *line = NULL;
    size_t len = 0;
    int warning = 0;

```

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.

C



```
while (1) {
    print_prs();
    prompt();
    fflush(stdout);

    ssize_t r = getline(&line, &len, stdin);
    if (r == -1) {
        if (!feof(stdin)) {
            /* Interrupted by Ctrl+C at prompt: clear error and redraw prompt */
            clearerr(stdin);
            warning = 0;
            continue;
        }

        /* Handles Ctrl+D (EOF on empty input line) */
        printf("\n");
        if (stopped() && !warning) {
            printf("cshell: there are stopped jobs\n");
            warning = 1;
            clearerr(stdin); /* Clear EOF flag so the next getline() blocks */
            continue;
        }

        /* Send SIGHUP to all active/stopped process groups on shell exit */
        send_sighup();
        break;
    }

    warning = 0; /* Reset warning state on valid non-empty input */
    /* Parse line and execute commands */
}
```

Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

```

    }

    /* Send SIGHUP to all active/stopped process groups on shell exit */
    send_sighup();
    break;
}

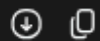
warning = 0; /* Reset warning state on valid non-empty input */
/* Parse line and execute commands */
}

free(line);
return 0;
}

```

Foreground Terminal Handoff & Suspend Handling (`execution.c`)

C



```

/* Child Process Execution Block (pid == 0) */
else if (pid == 0) {
    pid_t pgid = (c == 0) ? 0 : pids[0];
    setpgid(0, pgid);

    /* Reset terminal signals back to default for child execution */
    signal(SIGINT, SIG_DFL);
}

```

+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

```

    signal(SIGTSTP, SIG_DFL);
    signal(SIGTTIN, SIG_DFL);
    signal(SIGTTOU, SIG_DFL);

    /* Redirection and execv execution */
    execvp(p, arg);
    perror("execv error");
    exit(EXIT_FAILURE);
}

/* Parent Process Handling Block (pid > 0) */
else {
    pids[c] = pid;
    pid_t pgid = (c == 0) ? pid : pids[0];
    setpgid(pid, pgid);
}

/* Foreground Execution Block */
if (!bg) {
    /* Transfer terminal ownership to the child process group */
    tcsetpgrp(STDIN_FILENO, pids[0]);

    int is_stopped = 0;
    for (int c = 0; c < n; c++) {
        int status;
        if (pids[c] > 0) {
            waitpid(pids[c], &status, WUNTRACED);
            update_state(pids[c], status);

            /* Detect if process was suspended via Ctrl+Z */
            if (WIFSTOPPED(status)) {
                is_stopped = 1;
            }
        }
    }
}

```

+ Ask Gemini

Flash Extended ▼



Gemini is AI and can make mistakes.

```

}

/* Foreground Execution Block */
if (!bg) {
    /* Transfer terminal ownership to the child process group */
    tcsetpgrp(STDIN_FILENO, pids[0]);

    int is_stopped = 0;
    for (int c = 0; c < n; c++) {
        int status;
        if (pids[c] > 0) {
            waitpid(pids[c], &status, WUNTRACED);
            update_state(pids[c], status);

            /* Detect if process was suspended via Ctrl+Z */
            if (WIFSTOPPED(status)) {
                is_stopped = 1;
            }
        }
    }

    /* Reclaim terminal control for shell */
    tcsetpgrp(STDIN_FILENO, getpid());

    if (is_stopped) {
        printf("[%d] + Stopped    %s\n", job_id, cmd_str);
        fflush(stdout);
    }
}
}

```

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.

for c shell implment timeout feature for command resume
%job_number (fg [--timeout <seconds>] | bg)
when running job with timeout set timer for given seconds
if child finsihes before timer finishes, give control back to

C



```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>
#include <errno.h>

/* Global atomic flag for signal safety */
static volatile sig_atomic_t timed_out = 0;

/* Signal handler for SIGALRM */
static void sigalrm_handler(int sig) {
    (void) sig;
    timed_out = 1;
}
```

+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.


```

static void sigalrm_handler(int sig) {
    (void)sig;
    timed_out = 1;
}

void handle_fg_timeout(int job_i, int timeout) {
    printf("%s\n", jobs[job_i].cmd);
    fflush(stdout);

    /* 1. Transfer terminal control to foreground process group */
    tcsetpgrp(STDIN_FILENO, jobs[job_i].pgid);

    /* Mark active processes in job as running */
    for (int k = 0; k < jobs[job_i].num; k++) {
        if (jobs[job_i].prs[k].state != 2) {
            jobs[job_i].prs[k].state = 0;
        }
    }

    /* Send SIGCONT to resume target process group */
    kill(-jobs[job_i].pgid, SIGCONT);

    /* 2. Configure SIGALRM and arm timer if timeout > 0 */
    struct sigaction sa_old, sa_alm;
    if (timeout > 0) {
        timed_out = 0;
        sa_alm.sa_handler = sigalrm_handler;
        sigemptyset(&sa_alm.sa_mask);
        sa_alm.sa_flags = 0; // DO NOT set SA_RESTART
        sigaction(SIGALRM, &sa_alm, &sa_old);
        alarm(timeout); // Start countdown timer
    }
}

```

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.

C



```
int stopped_job = 0;

/* 3. Wait loop for process group */
for (int k = 0; k < jobs[job_i].num; k++) {
    if (jobs[job_i].prs[k].state == 2) continue; // Skip finished processes

    int status;
    pid_t res;
    while (1) {
        res = waitpid(jobs[job_i].prs[k].pid, &status, WUNTRACED);
        if (res > 0) {
            update_state(res, status);
            if (WIFSTOPPED(status)) stopped_job = 1;
            break;
        } else if (res < 0) {
            if (errno == EINTR) {
                /* Check if interrupted by SIGALRM timeout */
                if (timed_out) break;
                continue; /* Retry wait if interrupted by another signal */
            }
            break;
        }
    }
    if (timed_out) break; // Exit loop immediately if timer expired
}

/* 4. Cancel pending timer if job finishes or stops early */
if (timeout > 0) {
    alarm(0); // Disarm alarm
}
```

Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

```

/* 4. Cancel pending timer if job finishes or stops early */
if (timeout > 0) {
    alarm(0); // Disarm alarm
    sigaction(SIGALRM, &sa_old, NULL); // Restore original SIGALRM handler
}

/* 5. Handle timeout termination */
if (timed_out) {
    kill(-jobs[job_i].pgid, SIGTERM);
    printf("resume: job timed out\n");
    fflush(stdout);
    update_state(-1, 0); // Update process state in job table
}

/* 6. Reclaim terminal control back for the shell process group */
tcsetpgrp(STDIN_FILENO, getpid());

if (stopped_job && !timed_out) {
    printf("[%d] + Stopped    %s\n", jobs[job_i].job_id, jobs[job_i].cmd);
    fflush(stdout);
}

print_prs();
}

```

Step-by-Step Code Explanation

1. Atomic Flag & Signal Handler

+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

Step-by-Step Code Explanation

1. Atomic Flag & Signal Handler

- `static volatile sig_atomic_t timed_out: sig_atomic_t` ensures reads and writes to the variable are atomic (uninterruptible). `volatile` prevents the C compiler from caching the value in a CPU register, forcing it to re-read memory on every iteration.
- `sigalarm_handler()` : Runs when `SIGALRM` is delivered by the OS kernel when the timer hits zero. It sets `timed_out = 1`.

2. Terminal Handoff & Signal Setup

- `tcsetpgrp(STDIN_FILENO, jobs[job_i].pgid)` : Hands control of keyboard input (`stdin`) to the process group before resuming it.
- `sa_alrm.sa_flags = 0` : Setting `sa_flags = 0` without `SA_RESTART` ensures blocking system calls like `waitpid()` immediately return `-1` with `errno == EINTR` when `SIGALRM` strikes.
- `alarm(timeout)` : Asks the kernel to send `SIGALRM` to the shell after `timeout` seconds.

3. Interrupted Wait Loop

- While inside `waitpid()`, if `SIGALRM` fires, `waitpid()` breaks and returns `-1`.
- The `if (errno == EINTR)` check inspects `timed_out`. If `1`, it breaks out of the waiting loop immediately instead of resuming wait.

+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

- While inside `waitpid()`, if `SIGALRM` fires, `waitpid()` breaks and returns `-1`.
- The `if (errno == EINTR)` check inspects `timed_out`. If `1`, it breaks out of the waiting loop immediately instead of resuming wait.

4. Disarming the Timer

- `alarm(0)` : If the job exits or gets suspended before the timeout limit, `alarm(0)` cancels any remaining timer countdown so `SIGALRM` won't trigger later during shell prompt execution.
- `sigaction(SIGALRM, &sa_old, NULL)` restores the previous signal disposition.

5. Job Termination (`SIGTERM`)

- `kill(-jobs[job_i].pgid, SIGTERM)` : Using a negative process group ID (`-pgid`) sends the termination signal to **all** processes in the target group (e.g., all commands in a pipeline).
- `printf("resume: job timed out\n")` : Displays the required output.

6. Reclaiming Terminal Control

- `tcsetpgrp(STDIN_FILENO, getpid())` : Restores control of the terminal to the shell process group so user typing returns to `cshell` rather than the killed or suspended process.

👍 🗨️ ↻ 📄 ...

+ Ask Gemini

Flash Extended ▼



what are the different file types like REG,DIR etc and what function checks them

File types in Linux systems are checked using the `stat()` system call (or `lstat()` / `fstat()`) from `<sys/stat.h>`. The system call populates a `struct stat` structure, and POSIX macros test the `st_mode` bitmask field to identify the type.

Standard File Types and Check Macros

Abbreviation	File Type	Description	POSIX Macro
REG	Regular File	Standard data files (text files, compiled binaries, shared <code>.so</code> libraries).	<code>S_ISREG(st_mode)</code>
DIR	Directory	Folders containing directory entries.	<code>S_ISDIR(st_mode)</code>
CHR	Character Device	Unbuffered character-stream hardware or virtual devices (e.g., <code>/dev/tty</code>).	<code>S_ISCHR(st_mode)</code>

+ Ask Gemini

Flash Extended



CHR	Character Device	Unbuffered character-stream hardware or virtual devices (e.g., <code>/dev/pts/0</code> , <code>/dev/null</code>).	<code>S_ISCHR(st_e)</code>
BLK	Block Device	Buffered block-access storage devices (e.g., hard drives <code>/dev/sda</code> , disk partitions).	<code>S_ISBLK(st_e)</code>
FIFO	Named Pipe / FIFO	Inter-process communication channels created via <code>mkfifo</code> or shell pipes.	<code>S_ISFIFO(st_de)</code>
SOCK	Socket	Unix domain or network sockets created via <code>socket()</code> .	<code>S_ISSOCK(st_de)</code>
LNK	Symbolic Link	Soft links referencing another file path (checked via <code>lstat()</code>).	<code>S_ISLNK(st_e)</code>

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.

LNK

Symbolic Link

Soft links referencing
another file path
(checked via `lstat()`
).

`S_ISLNK(st.
e)`

...

How to Check Them in C

1. Call `stat(path, &st)` (or `lstat()` to avoid following symlinks).
2. Pass `st.st_mode` into the `S_IS*` macros.

C



```
#include <sys/stat.h>
#include <stdio.h>

const char* check_type(const char* path) {
    struct stat st;
    if (stat(path, &st) != 0) {
        return "UNKNOWN";
    }

    if (S_ISREG(st.st_mode)) return "REG";
    if (S_ISDIR(st.st_mode)) return "DIR";
    if (S_ISLNK(st.st_mode)) return "LNK";
    if (S_ISFIFO(st.st_mode)) return "FIFO";
    if (S_ISSOCK(st.st_mode)) return "SOCK";
    return "UNKNOWN";
}
```

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.

2. Pass `st.st_mode` into the `S_IS*` macros.

C



```
#include <sys/stat.h>
#include <stdio.h>

const char* check_type(const char* path) {
    struct stat st;
    if (stat(path, &st) != 0) {
        return "UNKNOWN";
    }

    if (S_ISREG(st.st_mode)) return "REG";
    if (S_ISDIR(st.st_mode)) return "DIR";
    if (S_ISCHR(st.st_mode)) return "CHR";
    if (S_ISBLK(st.st_mode)) return "BLK";
    if (S_ISFIFO(st.st_mode)) return "FIFO";
    if (S_ISSOCK(st.st_mode)) return "SOCK";
    if (S_ISLNK(st.st_mode)) return "LNK";

    return "REG";
}
```

Note: Virtual descriptors resolved via `/proc/<pid>/fd/` (like `pipe:[1234]` or `socket:[5678]`) will cause `stat()` to fail because they are internal kernel names, requiring a string check (`strcmp`) before calling `stat()` .

+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes

give a c string array of linux system call names match the indexes to official syscall numbers

```
const char* syscall_names[] = {
[0] = "read", [1] = "write", [2] = "open", [3] = "close", [4] = "stat",
[5] = "fstat", [6] = "lstat", [7] = "poll", [8] = "lseek", [9] = "mmap",
[10] = "mprotect", [11] = "munmap", [12] = "brk", [13] = "rt_sigaction",
[14] = "rt_sigprocmask", [15] = "rt_sigreturn", [16] = "ioctl", [17] = "pread64",
[18] = "pwrite64", [19] = "readv", [20] = "writev", [21] = "access",
[22] = "pipe", [23] = "select", [24] = "sched_yield", [25] = "mremap",
[26] = "msync", [27] = "mincore", [28] = "madvise", [29] = "shmget",
[30] = "shmat", [31] = "shmdt", [32] = "dup", [33] = "dup2", [34] = "pause",
[35] = "nanosleep", [36] = "getitimer", [37] = "alarm", [38] = "setitimer",
[39] = "getpid", [40] = "sendfile", [41] = "socket", [42] = "connect",
[43] = "accept", [44] = "sendto", [45] = "recvfrom", [46] = "sendmsg",
[47] = "recvmsg", [48] = "shutdown", [49] = "bind", [50] = "listen",
[51] = "getsockname", [52] = "getpeername", [53] = "socketpair", [54] = "setsockopt",
[55] = "getsockopt", [56] = "clone", [57] = "fork", [58] = "vfork",
[59] = "execve", [60] = "exit", [61] = "wait4", [62] = "kill", [63] = "uname",
[64] = "semget", [65] = "semop", [66] = "semctl", [67] = "shmctl",
[68] = "msgget", [69] = "msgsnd", [70] = "msgrcv", [71] = "msgctl",
[72] = "fcntl", [73] = "flock", [74] = "fsync", [75] = "fdatasync",
[76] = "truncate", [77] = "ftruncate", [78] = "getdents", [79] = "getcwd",
[80] = "chdir", [81] = "fchdir", [82] = "rename", [83] = "mkdir",
[84] = "rmdir", [85] = "creat", [86] = "link", [87] = "unlink",
```

+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

```
[64] = "semget", [65] = "semop", [66] = "semctl", [67] = "shmctl",  
[68] = "msgget", [69] = "msgsnd", [70] = "msgrcv", [71] = "msgctl",  
[72] = "fcntl", [73] = "flock", [74] = "fsync", [75] = "fdatasync",  
[76] = "truncate", [77] = "ftruncate", [78] = "getdents", [79] = "getcwd",  
[80] = "chdir", [81] = "fchdir", [82] = "rename", [83] = "mkdir",  
[84] = "rmdir", [85] = "creat", [86] = "link", [87] = "unlink",  
[88] = "symlink", [89] = "readlink", [90] = "chmod", [91] = "fchmod",  
[92] = "chown", [93] = "fchown", [94] = "lchown", [95] = "umask",  
[96] = "gettimeofday", [97] = "getrlimit", [98] = "getrusage", [99] = "sysinfo",  
[100] = "times", [101] = "ptrace", [102] = "getuid", [103] = "syslog",  
[104] = "getgid", [105] = "setuid", [106] = "setgid", [107] = "geteuid",  
[108] = "getegid", [109] = "setpgid", [110] = "getppid", [111] = "getpgrp",  
[112] = "setsid", [113] = "setreuid", [114] = "setregid", [115] = "getgroups",  
[116] = "setgroups", [117] = "setresuid", [118] = "getresuid", [119] = "setresgid",  
[120] = "getresgid", [121] = "getpgid", [122] = "setfsuid", [123] = "setfsgid",  
[124] = "getsid", [125] = "capget", [126] = "capset", [127] = "rt_sigpending",  
[128] = "rt_sigtimedwait", [129] = "rt_sigqueueinfo", [130] = "rt_sigsuspend",  
[131] = "sigaltstack", [132] = "utime", [133] = "mknod", [134] = "uselib",  
[135] = "personality", [136] = "ustat", [137] = "statfs", [138] = "fstatfs",  
[139] = "sysfs", [140] = "getpriority", [141] = "setpriority", [142] = "sched_setparam",  
[143] = "sched_getparam", [144] = "sched_setscheduler", [145] = "sched_getscheduler",  
[146] = "sched_get_priority_max", [147] = "sched_get_priority_min",  
[148] = "sched_rr_get_interval", [149] = "mlock", [150] = "munlock",  
[151] = "mlockall", [152] = "munlockall", [153] = "vhangup", [154] = "modify_ldt",  
[155] = "pivot_root", [156] = "_sysctl", [157] = "prctl", [158] = "arch_prctl",  
[159] = "adjtimex", [160] = "setrlimit", [161] = "chroot", [162] = "sync",  
[163] = "acct", [164] = "settimeofday", [165] = "mount", [166] = "umount2",  
[167] = "swapon", [168] = "swapoff", [169] = "reboot", [170] = "sethostname",
```

+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.

```
[243] = "mq_timedreceive", [244] = "mq_notify", [245] = "mq_getsetattr",
[246] = "kexec_load", [247] = "waitid", [248] = "add_key", [249] = "request_key",
[250] = "keyctl", [251] = "ioprio_set", [252] = "ioprio_get", [253] = "inotify_init",
[254] = "inotify_add_watch", [255] = "inotify_rm_watch", [256] = "migrate_pages",
[257] = "openat", [258] = "mkdirat", [259] = "mknodat", [260] = "fchownat",
[261] = "futimesat", [262] = "newfstatat", [263] = "unlinkat", [264] = "renameat",
[265] = "linkat", [266] = "symlinkat", [267] = "readlinkat", [268] = "fchmodat",
[269] = "faccessat", [270] = "pselect6", [271] = "ppoll", [272] = "unshare",
[273] = "set_robust_list", [274] = "get_robust_list", [275] = "splice",
[276] = "tee", [277] = "sync_file_range", [278] = "vmsplice", [279] = "move_pages",
[280] = "utimensat", [281] = "epoll_pwait", [282] = "signalfd",
[283] = "timerfd_create", [284] = "eventfd", [285] = "fallocate",
[286] = "timerfd_settime", [287] = "timerfd_gettime", [288] = "accept4",
[289] = "signalfd4", [290] = "eventfd2", [291] = "epoll_create1", [292] = "dup3",
[293] = "pipe2", [294] = "inotify_init1", [295] = "preadv", [296] = "pwritev",
[297] = "rt_tgsigqueueinfo", [298] = "perf_event_open", [299] = "recvmmsg",
[300] = "fanotify_init", [301] = "fanotify_mark", [302] = "prlimit64",
[303] = "name_to_handle_at", [304] = "open_by_handle_at", [305] = "clock_adjtime",
[306] = "syncfs", [307] = "sendmmsg", [308] = "setns", [309] = "getcpu",
[310] = "process_vm_readv", [311] = "process_vm_writev", [312] = "kcmp",
[313] = "finit_module", [314] = "sched_setattr", [315] = "sched_getattr",
[316] = "renameat2", [317] = "seccomp", [318] = "getrandom", [319] = "memfd_create",
[320] = "kexec_file_load", [321] = "bpf", [322] = "execveat", [323] = "userfaultfd",
[324] = "membarrier", [325] = "mlock2", [326] = "copy_file_range",
[327] = "preadv2", [328] = "pwritev2", [329] = "pkey_mprotect", [330] = "pkey_alloc",
[331] = "pkey_free", [332] = "statx", [333] = "io_pgetevents", [334] = "rseq"
};
```

👍 💬 📄 ...

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.

Purpose: snoop traces a process's syscalls and reports a summary once it exits.

Requirements

If a command is given, the shell must fork, call



Here is the complete implementation of `snoop` tailored for your C shell architecture. It handles process creation/attachment, system call entry/exit tracking, high-precision timing, sorting, and error handling according to all requirements.

Implementation (`snoop.c`)

c



```
#define _DEFAULT_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <time.h>
#include <errno.h>
#include <signal.h>
#include <sys/ptrace.h>
#include <sys/user.h>
#include <sys/wait.h>
#include <sys/stat.h>
```

+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

```

extern const char* syscall_names[];

typedef struct {
    int num;
    char name[256];
    int count;
    double time;
    int first_seen; // Tracks original order for tie-breaking
} SyscallStats;

// Sort by call count descending; break ties by order of first occurrence
static int compare_stats(const void* a, const void* b) {
    const SyscallStats* s1 = (const SyscallStats*)a;
    const SyscallStats* s2 = (const SyscallStats*)b;
    if (s1->count != s2->count) {
        return s2->count - s1->count;
    }
    return s1->first_seen - s2->first_seen;
}

// Fallback lookup: Returns syscall name or "syscall_N" if missing/out-of-range
static void resolve_syscall_name(int num, char* out_buf, size_t buf_size) {
    int table_size = (int)(sizeof(syscall_names) / sizeof(syscall_names[0]));
    if (num >= 0 && num < table_size && syscall_names[num] != NULL) {
        snprintf(out_buf, buf_size, "%s", syscall_names[num]);
    } else {
        snprintf(out_buf, buf_size, "syscall_%d", num);
    }
}

```

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.

```

11 (strlen(cmd), / ) := NULL) {
    if (access(cmd, X_OK) == 0) {
        snprintf(resolved_path, path_size, "%s", cmd);
        return 1;
    }
    return 0;
}

char* path_env = getenv("PATH");
if (!path_env) path_env = "/bin:/usr/bin";

char path_copy[2048];
strncpy(path_copy, path_env, sizeof(path_copy) - 1);
path_copy[sizeof(path_copy) - 1] = '\0';

char* dir = strtok(path_copy, ":");
while (dir != NULL) {
    snprintf(resolved_path, path_size, "%s/%s", dir, cmd);
    if (access(resolved_path, X_OK) == 0) {
        return 1;
    }
    dir = strtok(NULL, ":");
}
return 0;
}

void snoop(char** input, int count) {
    if (count < 2) {
        printf("snoop: invalid syntax\n");
        fflush(stdout);
        return;
    }
}

```

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.


```

// --- Mode 1: Attach to PID (-p pid) ---
if (strcmp(input[1], "-p") == 0) {
    if (count != 3 || input[2] == NULL || input[2][0] == '\0') {
        printf("snoop: invalid syntax\n");
        fflush(stdout);
        return;
    }
    for (int i = 0; input[2][i] != '\0'; i++) {
        if (input[2][i] < '0' || input[2][i] > '9') {
            printf("snoop: no such process\n");
            fflush(stdout);
            return;
        }
    }
    pid = (pid_t)atoi(input[2]);
    if (pid <= 0 || ptrace(PTRACE_ATTACH, pid, NULL, NULL) == -1) {
        printf("snoop: no such process\n");
        fflush(stdout);
        return;
    }
}

// --- Mode 2: Fork and Execve New Command ---
else {
    char resolved_path[1024];
    if (!resolve_command(input[1], resolved_path, sizeof(resolved_path))) {
        printf("snoop: command not found\n");
        fflush(stdout);
        return;
    }
}

```

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.

```

sigaddset(&mask, SIGCHLD);
sigprocmask(SIG_BLOCK, &mask, &oldmask);

int status;
if (waitpid(pid, &status, 0) == -1) {
    sigprocmask(SIG_SETMASK, &oldmask, NULL);
    return;
}

// Set TRACESYSGOOD so system call stops deliver (SIGTRAP | 0x80)
ptrace(PTRACE_SETOPTIONS, pid, 0, (void*)(long)PTRACE_O_TRACESYSGOOD);

static SyscallStats stats[2048];
int stats_count = 0;
int in_syscall = 0;
int curr_syscall = -1;
struct timespec start_time;

// Start system call tracing loop
ptrace(PTRACE_SYSCALL, pid, 0, 0);

while (1) {
    if (waitpid(pid, &status, 0) == -1) {
        break;
    }

    // Tracee exit detection
    if (WIFEXITED(status) || WIFSIGNALED(status)) {
        if (in_syscall) {
            struct timespec end_time;
            clock_gettime(CLOCK_MONOTONIC, &end_time);

```

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.

C

```
----- syscall_stats -----;
clock_gettime(CLOCK_MONOTONIC, &end_time);
double elapsed = (end_time.tv_sec - start_time.tv_sec) +
    (end_time.tv_nsec - start_time.tv_nsec) / 1e9;

int found = 0;
for (int i = 0; i < stats_count; i++) {
    if (stats[i].num == curr_syscall) {
        stats[i].count++;
        stats[i].time += elapsed;
        found = 1;
        break;
    }
}
if (!found && stats_count < 2048) {
    stats[stats_count].num = curr_syscall;
    resolve_syscall_name(curr_syscall, stats[stats_count].name, sizeof(stats[stats_count].name));
    stats[stats_count].count = 1;
    stats[stats_count].time = elapsed;
    stats[stats_count].first_seen = stats_count;
    stats_count++;
}
in_syscall = 0;
}
break;
}

// Tracee stopped state handler
if (WIFSTOPPED(status)) {
    int sig = WSTOPSIG(status);
```

[+ Ask Gemini](#)

Flash Extended ▾



Gemini is AI and can make mistakes.

```

struct user_regs_struct regs;
if (ptrace(PTRACE_GETREGS, pid, 0, &regs) == 0) {
    if (in_syscall == 0) {
        // Syscall Entry
        curr_syscall = (int)regs.orig_rax;
        clock_gettime(CLOCK_MONOTONIC, &start_time);
        in_syscall = 1;
    } else {
        // Syscall Exit
        struct timespec end_time;
        clock_gettime(CLOCK_MONOTONIC, &end_time);
        double elapsed = (end_time.tv_sec - start_time.tv_sec) +
            (end_time.tv_nsec - start_time.tv_nsec) / 1e9;

        int found = 0;
        for (int i = 0; i < stats_count; i++) {
            if (stats[i].num == curr_syscall) {
                stats[i].count++;
                stats[i].time += elapsed;
                found = 1;
                break;
            }
        }
        if (!found && stats_count < 2048) {
            stats[stats_count].num = curr_syscall;
            resolve_syscall_name(curr_syscall, stats[stats_count].name);
            stats[stats_count].count = 1;
            stats[stats_count].time = elapsed;
            stats[stats_count].first_seen = stats_count;
            stats_count++;
        }
    }
}

```

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.

```

        in_syscall = 0;
    }
}
ptrace(PTRACE_SYSCALL, pid, 0, 0);
} else if (sig == SIGTRAP) {
    ptrace(PTRACE_SYSCALL, pid, 0, 0);
} else {
    // Forward standard signals to tracee
    ptrace(PTRACE_SYSCALL, pid, 0, (void*)(long)sig);
}
}
}

// Restore signal mask
sigprocmask(SIG_SETMASK, &oldmask, NULL);

// Sort: descending by count, ascending by first_seen
qsort(stats, stats_count, sizeof(SyscallStats), compare_stats);

// Output formatted summary table
if (stats_count > 0) {
    printf("%-14s%-8s\n", "syscall", "calls", "time");
    for (int i = 0; i < stats_count; i++) {
        printf("%-14s%-8d%.3fs\n", stats[i].name, stats[i].count, stats[i].time);
    }
}
fflush(stdout);
}

```

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.

```

// Output formatted summary table
if (stats_count > 0) {
    printf("%-14s%-8s\n", "syscall", "calls", "time");
    for (int i = 0; i < stats_count; i++) {
        printf("%-14s%-8d%.3fs\n", stats[i].name, stats[i].count, stats[i].time);
    }
}
fflush(stdout);
}

```

Key Architectural Highlights

1. **State Machine (`in_syscall`):** Toggles back and forth between 0 (Entry) and 1 (Exit) on `(SIGTRAP | 0x80)` stops. Entry captures `orig_rax` and `CLOCK_MONOTONIC` start time; Exit computes elapsed time and updates aggregated stats.
2. **Signal & Mask Management:** Temporarily blocks `SIGCHLD` via `sigprocmask` so shell background process cleanup handlers do not interrupt `waitpid()` tracing loops. Signal forwarding ensures tracees receiving signals continue tracing seamlessly.
3. **Lookup & Fallback:** Checks indices against array bounds; formats missing entries dynamically as `syscall_N`.
4. **Stable Tie-Breaking:** `first_seen` records chronological registration order, enabling `qsort` to break count ties based on first occurrence.

👍 🗨️ ↻ 📄 ...

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.

for xv6 project make makefile to support different scheduling algorithms based on a "SCHEDULER" variable, it must default to RR if not specified when running make, it should support RR, FCFS and MLFQ, add relevant flags, standard rules for project like this, i will be creating a scheduler test for testing so it should also be incorporated

Makefile

```
# Scheduling Policy Selection
# Options: RR (default), FCFS, MLFQ
SCHEDULER ?= RR

ifeq ($(SCHEDULER), FCFS)
CFLAGS += -DSCHEDULER_FCFS
else ifeq ($(SCHEDULER), MLFQ)
CFLAGS += -DSCHEDULER_MLFQ
else ifeq ($(SCHEDULER), RR)
CFLAGS += -DSCHEDULER_RR
else
$(error Invalid SCHEDULER flag '$(SCHEDULER)'. Supported options: RR, FCFS, MLFQ)
endif

# Toolchain selection
K=kernel
```

+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

U=user

```
OBJS = \  
$K/entry.o \  
$K/start.o \  
$K/console.o \  
$K/printf.o \  
$K/uart.o \  
$K/kalloc.o \  
$K/spinlock.o \  
$K/string.o \  
$K/main.o \  
$K/vm.o \  
$K/proc.o \  
$K/swtch.o \  
$K/trampoline.o \  
$K/trap.o \  
$K/syscall.o \  
$K/sysproc.o \  
$K/bio.o \  
$K/fs.o \  
$K/log.o \  
$K/sleeplock.o \  
$K/file.o \  
$K/pipe.o \  
$K/exec.o \  
$K/sysfile.o \  
$K/kernelvec.o \  
$K/plic.o \  
$K/virtio_disk.o
```

+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

Makefile

```
TOOLPREFIX := $(shell if riscv64-unknown-elf-objdump -i 2>&1 | grep 'elf64-big' >/dev/null; \
then echo "riscv64-unknown-elf-"; \
elif riscv64-linux-gnu-objdump -i 2>&1 | grep 'elf64-big' >/dev/null 2>&1; \
then echo "riscv64-linux-gnu-"; \
elif riscv64-unknown-linux-gnu-objdump -i 2>&1 | grep 'elf64-big' >/dev/null; \
then echo "riscv64-unknown-linux-gnu-"; \
else echo "" ; fi)

endif

CC = $(TOOLPREFIX)gcc
AS = $(TOOLPREFIX)gas
LD = $(TOOLPREFIX)ld
OBJCOPY = $(TOOLPREFIX)objcopy
OBJDUMP = $(TOOLPREFIX)objdump

CFLAGS += -Wall -Werror -O -fno-omit-frame-pointer -ggdb -gdwarf-2
CFLAGS += -MD
CFLAGS += -mmodel=medany
CFLAGS += -ffreestanding -fno-common -nostdlib -mno-relax
CFLAGS += -I.
CFLAGS += $(shell $(CC) -fno-stack-protector -E -x c /dev/null >/dev/null 2>&1 && echo)

# Disable PIE for security-enabled GCC targets
ifneq ($(shell $(CC) -dumpspecs 2>/dev/null | grep -e '^[f]no-pie'),)
CFLAGS += -fno-pie -no-pie
endif
ifneq ($(shell $(CC) -dumpspecs 2>/dev/null | grep -e '^[f]nopie'),)
CFLAGS += -fno-pie -no-pie
endif

LDFLAGS = -z max-page-size=4096
```

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.

```

$(CC) $(CFLAGS) -march=rv64g -nostdinc -I. -Ikernel -c $U/initcode.S -o $U/
$(LD) $(LDFLAGS) -N -e start -Ttext 0 -o $U/initcode.out $U/initcode.o
$(OBJCOPY) -S -O binary $U/initcode.out $U/initcode
$(OBJDUMP) -S $U/initcode.o > $U/initcode.asm

tags: $(OBS) _init
      etags *.S *.c

ULIB = $U/ulib.o $U/usys.o $U/printf.o $U/umalloc.o

_%: %.o $(ULIB)
    $(LD) $(LDFLAGS) -T $U/user.ld -o $@ $^
    $(OBJDUMP) -S $@ > $.asm
    $(OBJDUMP) -t $@ | sed '1,/SYMBOL TABLE/d; /^[0-9a-fA-F]/!d; /$$/d' | sort

$U/usys.S : $U/usys.pl
    perl $U/usys.pl > $U/usys.S

$U/usys.o : $U/usys.S
    $(CC) $(CFLAGS) -c -o $U/usys.o $U/usys.S

$U/_forktest: $U/forktest.o $(ULIB)
    # forktest has less code than the others, so it can be linked
    # lower in the address space.
    $(LD) $(LDFLAGS) -N -e main -Ttext 0 -o $U/_forktest $U/forktest.o $(ULIB)
    $(OBJDUMP) -S $U/_forktest > $U/forktest.asm

# Added $U/_schedulertest to user program target list
UPROGS=\
    $U/_cat\
    $U/_echo\

```

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.

```

rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
  **/*.o **/*.d **/*.asm **/*.sym \
  $U/initcode $U/initcode.out $K/kernel fs.img \
  mkfs/mkfs .gdbinit \
  $U/usys.S \
  $(UPROGS)

# QEMU configuration
ifndef QEMU
QEMU = qemu-system-riscv64
endif

CPUS := 1

QEMUOPTS = -machine virt -bios none -kernel $K/kernel -m 128M -smp $(CPUS) -nographic
QEMUOPTS += -global virtio-mmio.force-raw=false
QEMUOPTS += -drive file=fs.img,if=none,format=raw,id=x0
QEMUOPTS += -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

qemu: $K/kernel fs.img
    $(QEMU) $(QEMUOPTS)

.gdbinit: .gdbinit.tmpl-riscv
    sed "s/:1234/:$(GDBPORT)/" < $^ > $@

qemu-gdb: $K/kernel .gdbinit fs.img
    @echo "*** Now run 'gdb' in another window." 1>&2
    $(QEMU) $(QEMUOPTS) -S -gdb tcp::$(GDBPORT)

```

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.

Makefile

```
QEMUOPTS = -machine virt -bios none -kernel $K/kernel -m 128M -smp $(CPUS) -nographic
QEMUOPTS += -global virtio-mmio.force-raw=false
QEMUOPTS += -drive file=fs.img,if=none,format=raw,id=x0
QEMUOPTS += -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

qemu: $K/kernel fs.img
    $(QEMU) $(QEMUOPTS)

.gdbinit: .gdbinit.tmpl-riscv
    sed "s/:1234/:$(GDBPORT)/" < $^ > $@

qemu-gdb: $K/kernel .gdbinit fs.img
    @echo "*** Now run 'gdb' in another window." 1>&2
    $(QEMU) $(QEMUOPTS) -S -gdb tcp::$(GDBPORT)
```

Key Modifications Explained

- `SCHEDULER ?= RR` : Sets the default scheduler macro to Round Robin (`RR`) if `SCHEDULER` is omitted on the command line.
- **Conditional Macro Definition**: Uses Makefile conditionals (`ifeq`) to inject `-DSCHEDULER_RR` , `-DSCHEDULER_FCFS` , or `-DSCHEDULER_MLFQ` into `CFLAGS` .
- **Guard against invalid flags**: Raises a Makefile `$(error ...)` if an unsupported algorithm is passed.
- `$U/_schedulertest` : Automatically compiles `user/schedulertest.c` and builds it into `fs.img` .

+ Ask Gemini

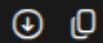
Flash Extended ▾



Gemini is AI and can make mistakes.

write c code for process queue ordering and mlfq priority boosting inside kernel scheduler
use a helper that assigns global sequence numbers increasing monotonically when it enters the queue
in mlfq use priority boost to avoid starvation of processes
ignore the unused and zombie states, use the above helper for the runnable state
all this is to be done only if MLFQ/FCFS are defined

C



```
#if defined(SCHED_MLFQ) || defined(SCHED_FCFS)

// Global sequence counter and lock to maintain arrival/FIFO ordering
static uint64 next_sequence = 1;
static struct spinlock sequence_lock;

// Assigns a monotonically increasing sequence number to a process on queue insertion
static void enqueue_assign_sequence(struct proc *p) {
    acquire(&sequence_lock);
    p->sequence = next_sequence++;
    release(&sequence_lock);
}

#endif SCHED_MLFQ
```

+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

```

#ifdef SCHED_MLFQ
// Priority Boost: Restores all active processes to Queue 0 to prevent starvation
void mlfq_priority_boost(void) {
    struct proc *p;

    for (p = proc; p < &proc[NPROC]; p++) {
        acquire(&p->lock);

        // Skip inactive and terminated processes
        if (p->state != UNUSED && p->state != ZOMBIE) {
            p->queue_prio = 0;        // Move back to highest priority queue (Queue 0)
            p->ticks_used = 0;        // Reset consumed time-slice ticks
            p->last_boost = ticks;    // Record the boost timestamp

            // Assign a new arrival sequence number if currently runnable
            if (p->state == RUNNABLE) {
                enqueue_assign_sequence(p);
            }
        }

        release(&p->lock);
    }
}
#endif
#endif

```

Want to see the corresponding proc struct changes for sequence tracking?

+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

write c code for process selection logic in xv6 scheduler supporting MLFQ,FCFS via macros and RR as default
MLFQ: leave some empty space for selecting the process
implement acquiring and releasing locks during the scan
FCFS:
leave empty space for process selection
use same lock pattern here
RR(default)
leave empty space for process selection

use context switch using the appropriate function, and handle the interrupts

C



```
#ifndef SCHEDULER_MLFQ
for (int queue = 0; queue < MLFQ_LEVELS && !found; queue++) {
    struct proc *selected = 0;

    for (p = proc; p < &proc[NPROC]; p++) {
        acquire(&p->lock);

        /* =====
        * TODO: Process Selection Logic (MLFQ)
        * Find the process with highest priority for current queue level
        */
    }
}
```

+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

```

    * Evaluate process eligibility for current queue level.
    * ===== */
if (/* ... selection condition for queue ... */) {
    if (selected != 0)
        release(&selected->lock); // Release previously held candidate lock
    selected = p;

    /* ... update tracking variables ... */

    continue; // Retain lock on newly selected process and continue scan
}

release(&p->lock);
}

if (selected != 0) {
    p = selected;
    p->state = RUNNING;

    if (!p->started) {
        p->started = 1;
        p->first_run = ticks;
    }

    c->proc = p;
    swtch(&c->context, &p->context);

    // Reset CPU interrupt state after returning from context switch
    mycpu()->intena = 0;
    c->proc = 0;
    found = 1;
}

```

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.


```

    }

    #elif defined(SCHEDULER_FCFS)
        struct proc *selected = 0;

        for (p = proc; p < &proc[NPROC]; p++) {
            acquire(&p->lock);

            /* =====
             * TODO: Process Selection Logic (FCFS)
             * Evaluate process eligibility based on arrival order.
             * ===== */
            if (/* ... selection condition for FCFS ... */) {
                if (selected != 0)
                    release(&selected->lock); // Release previously held candidate lock
                selected = p;

                /* ... update tracking variables ... */

                continue; // Retain lock on newly selected process and continue scan
            }

            release(&p->lock);
        }

        if (selected != 0) {
            p = selected;
            p->state = RUNNING;

            if (!p->started) {
                p->started = 1;
            }
        }
    }
}

```

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes

```

    // Reset CPU interrupt state after returning from context switch
    mycpu()->intena = 0;
    c->proc = 0;
    found = 1;
    release(&p->lock);
}

#else
// Default: Round-Robin (RR)
for (p = proc; p < &proc[NPROC]; p++) {
    acquire(&p->lock);

    /* =====
     * TODO: Process Selection Logic (Round-Robin)
     * Check if process is ready to run.
     * ===== */
    if (/* ... selection condition for RR ... */) {
        p->state = RUNNING;

        if (!p->started) {
            p->started = 1;
            p->first_run = ticks;
        }

        c->proc = p;
        swtch(&c->context, &p->context);

        // Reset CPU interrupt state after returning from context switch
        mycpu()->intena = 0;
        c->proc = 0;
        found = 1;
    }
}

```

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.

```

    acquire(&p->lock);

    /* =====
    * TODO: Process Selection Logic (Round-Robin)
    * Check if process is ready to run.
    * ===== */
    if (/* ... selection condition for RR ... */) {
        p->state = RUNNING;

        if (!p->started) {
            p->started = 1;
            p->first_run = ticks;
        }

        c->proc = p;
        swtch(&c->context, &p->context);

        // Reset CPU interrupt state after returning from context switch
        mycpu()->intena = 0;
        c->proc = 0;
        found = 1;
    }

    release(&p->lock);
}
#endif

```

👍 💬 ↻ 📄 ...

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.

add new xv6 system call it allows user to get scheduler's statistics and copy them into a file the call needs pointer to destination use other syscall handlers in kernel/sysproc.c for this

To add a new system call (`schedstats`) in xv6 RISC-V that extracts scheduler statistics from kernel space and copies them to a user-provided buffer address, update the kernel and user interface files as outlined below.

1. Shared Data Structure (`kernel/schedstat.h`)

Create a header defining the metrics structure shared between kernel and user space:

C

```
#ifndef SCHEDSTAT_H
#define SCHEDSTAT_H

struct sched_stat {
    int pid;
    int queue;           // Current queue level (0-3 for MLFQ)
    int ticks_used;      // Ticks consumed in current time slice
    int total_ticks;     // Total CPU ticks used over process lifespan
};
```

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.

```
int total_ticks;    // Total CPU ticks used over process lifespan
int arrival_time;   // Tick timestamp when process was created
int first_run_time; // Tick timestamp when process first scheduled
int end_time;       // Tick timestamp when process completed
};

#endif
```

2. System Call Handler (kernel/sysproc.c)

Add the system call handler following standard argument-retrieval patterns used in `sysproc.c` :

```
C

uint64
sys_schedstats(void)
{
    uint64 addr;

    // Extract user-space buffer pointer passed as argument 0
    argaddr(0, &addr);

    return getschedstats(addr);
}
```

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.

3. Kernel Helper Implementation (`kernel/proc.c`)

Implement `getschedstats` to safely fetch process bookkeeping fields under lock protection and copy them into user memory via `copyout` :

```
C

#include "schedstat.h"

int
getschedstats(uint64 addr)
{
    struct proc *p = myproc();
    struct sched_stat st;

    if (addr == 0)
        return -1;

    // Safely copy process metrics under lock
    acquire(&p->lock);
    st.pid = p->pid;
    st.queue = p->queue_prio;
    st.ticks_used = p->ticks_used;
    st.total_ticks = p->total_ticks;
    st.arrival_time = p->arrival_time;
    st.first_run_time = p->first_run_time;
    st.end_time = p->end_time;
```

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.

```
}
```

Add the function prototype to `kernel/defs.h`:

C

```
// proc.c
int getschedstats(uint64);
```

4. Wire System Call into Kernel Vector

`kernel/syscall.h`

Define the system call number:

C

```
#define SYS_schedstats 22
```

`kernel/syscall.c`

+ Ask Gemini

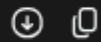
Flash Extended



Gemini is AI and can make mistakes.

Register the handler function:

C



```
extern uint64 sys_schedstats(void);

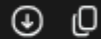
static uint64 (*syscalls[])(void) = {
    // ... existing system calls ...
    [SYS_schedstats]    sys_schedstats,
};
```

5. User-Space Bindings

user/usys.pl

Add the assembly stub generator entry:

Perl



```
entry("schedstats");
```

user/user.h

+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

Expose the function prototype and include the structure definition for user applications:

C



```
struct sched_stat;  
int schedstats(struct sched_stat*);
```

6. User Space Usage Example (user/statstest.c)

C



```
#include "kernel/types.h"  
#include "kernel/stat.h"  
#include "user/schedstat.h"  
#include "user/user.h"  
  
int  
main(void)  
{  
    struct sched_stat st;  
  
    if (schedstats(&st) < 0) {  
        printf("schedstats failed\n");  
        exit(1);  
    }  
}
```

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.

create custom xv6 program schedulertest.c to get scheduler's behaviour. it should fork child processes with different gaps lengths, including an I/O like workload that does short cpu bursts followed by small pauses, print each child's pid, progress, collect its statistics before exit and report the arrival,response,completion,turnaround, waitning,and cpu usage etcc

Here is the complete implementation of `user/schedulertest.c` designed to test and benchmark xv6 CPU schedulers (MLFQ, FCFS, and Round Robin).

It spawns child processes with staggered arrival times, CPU-bound workloads, and I/O-bound workloads (short bursts with `sleep()` pauses). Each child queries its scheduler stats via `schedstats()` right before exiting and reports its metrics.

`user/schedulertest.c`

c



+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

```

#include "kernel/types.h"
#include "user/user.h"

static void
cpu_burst(int ticks)
{
    volatile uint64 value = 0;
    uint start = uptime();
#ifdef SCHEDULER_MLFQ
    uint last_event_tick = start;
#endif

    while (uptime() - start < ticks) {
        value = value * 1664525 + 1013904223;
#ifdef SCHEDULER_MLFQ
        struct schedstats stats;
        uint now = uptime();
        if (now != last_event_tick && schedstats(&stats) == 0) {
            printf("event tick=%d pid=%d queue=%d\n", now, getpid(), stats.queue);
            last_event_tick = now;
        }
#endif
    }
    printf("pid %d cpu burst %d ticks complete\n", getpid(), ticks);
}

```

static void

+ Ask Gemini

Flash Extended



```

1
for (int round = 0; round < rounds; round++) {
    cpu_burst(2);
    pause(4);
}
printf("pid %d io workload complete\n", getpid());
}

static void
print_metrics(void)
{
    struct schedstats stats;
    int finish = uptime();

    if (schedstats(&stats) < 0) {
        printf("pid %d metrics unavailable\n", getpid());
        return;
    }
    printf("metrics pid=%d arrival=%d first=%d finish=%d turnaround=%d "
           "waiting=%d response=%d cpu=%d\n",
           getpid(), (int)stats.arrival, (int)stats.first_run, finish,
           (int)(finish - stats.arrival), (int)stats.waiting,
           (int)(stats.first_run - stats.arrival),
           (int)stats.ticks);
}

```

```
int
```

+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

```
int
main(void)
{
    int pid;

    printf("scheduler test start at tick %d\n", uptime());
    for (int workload = 0; workload < 4; workload++) {
        pid = fork();
        if (pid < 0) {
            printf("scheduler test: fork failed\n");
            exit(1);
        }
        if (pid == 0) {
            if (workload == 0)
                cpu_burst(12);
            else if (workload == 1)
                cpu_burst(28);
            else if (workload == 2)
                cpu_burst(56);
            else
                io_bound(8);
            print_metrics();
            exit(0);
        }
    }
}
```

+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

```
    printf("scheduler test: fork failed\n");
    exit(1);
}
if (pid == 0) {
    if (workload == 0)
        cpu_burst(12);
    else if (workload == 1)
        cpu_burst(28);
    else if (workload == 2)
        cpu_burst(56);
    else
        io_bound(8);
    print_metrics();
    exit(0);
}
}

while (wait(0) >= 0)
    ;
printf("scheduler test complete at tick %d\n", uptime());
exit(0);
}
```

👍 🗨️ ↻ 📄 ...

+ Ask Gemini

Flash Extended ▾



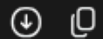
Gemini is AI and can make mistakes.

create python program that reads scheduler results from csv file that has timer tick, process id and queue number. generate timeline/scatter plot with timer ticks on x axis and MLFQ queueus 0 to 3 on y axis, use diff colors for each process, show priority boost in the plot add watermark 'vardaan.goel' in top right of plot accept an input csv and output the generated plot as png

Here is the complete Python program using `pandas` and `matplotlib` to parse your scheduler output CSV and generate the watermarked MLFQ timeline scatter plot.

`plot_mlfq_timeline.py`

Python



```
import argparse
import sys
import matplotlib.pyplot as plt
import pandas as pd
```

+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

```

def generate_mlfq_plot(csv_file: str, output_image: str):
    # Read CSV file
    try:
        df = pd.read_csv(csv_file)
    except Exception as e:
        print(f"Error reading {csv_file}: {e}")
        sys.exit(1)

    # Normalize column names (handles variations like 'timer tick', 'tick', 'process')
    df.columns = df.columns.str.strip().str.lower()
    column_mapping = {}

    for col in df.columns:
        if "tick" in col:
            column_mapping[col] = "tick"
        elif "pid" in col or "process" in col:
            column_mapping[col] = "pid"
        elif "queue" in col:
            column_mapping[col] = "queue"

    df = df.rename(columns=column_mapping)

    required_cols = {"tick", "pid", "queue"}
    if not required_cols.issubset(df.columns):
        print(
            f"Error: CSV must contain tick, pid, and queue columns. Found: {list(df.columns)}"
        )

```

+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

Python

```
sys.exit(1)

df = df.sort_values(by="tick")

# Set up plot
fig, ax = plt.subplots(figsize=(12, 6))

unique_pids = df["pid"].unique()
colors = plt.cm.tab10.colors

# Plot each process with discrete scatter points and trend line
for i, pid in enumerate(unique_pids):
    pid_data = df[df["pid"] == pid]
    color = colors[i % len(colors)]

    ax.scatter(
        pid_data["tick"],
        pid_data["queue"],
        label=f"PID {pid}",
        color=color,
        s=25,
        alpha=0.8,
        zorder=3,
    )
    ax.plot(
        pid_data["tick"],
```

+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

```

        alpha=0.3,
        linewidth=1.2,
        zorder=2,
    )

    # Highlight periodic priority boost lines (every 48 ticks)
    max_tick = df["tick"].max() if not df.empty else 150
    boost_ticks = range(48, int(max_tick) + 1, 48)

    for b in boost_ticks:
        ax.axvline(
            x=b,
            color="purple",
            linestyle="--",
            alpha=0.6,
            linewidth=1.5,
            label="Priority Boost (48 ticks)" if b == 48 else "",
            zorder=1,
        )

    # Axes and visual styling
    ax.set_ylim(-0.5, 3.5)
    ax.set_yticks([0, 1, 2, 3])
    ax.set_yticklabels(
        ["Q0 (Highest)", "Q1", "Q2", "Q3 (Lowest)"]
    ) # Queue 0 at top
    ax.invert_yaxis()

```

+ Ask Gemini

Flash Extended ▾



Gemini is AI and can make mistakes.

Python



```
ax.grid(True, linestyle=":", alpha=0.6, zorder=0)
ax.legend(loc="lower right", frameon=True, facecolor="white", framealpha=0.9)

# MANDATORY WATERMARK LOGIC
plt.text(
    0.95,
    0.95,
    "vardaan.goel",
    ha="right",
    va="top",
    transform=ax.transAxes,
    fontsize=10,
    color="gray",
    alpha=0.7,
)

plt.tight_layout()
plt.savefig(output_image, dpi=300)
print(f"Successfully saved MLFQ timeline plot to: {output_image}")

if __name__ == "__main__":
    parser = argparse.ArgumentParser(
        description="Generate MLFQ scheduler timeline plot from CSV."
    )
    parser.add_argument("input_csv", help="Path to input CSV file")
```

+ Ask Gemini

Flash Extended



Gemini is AI and can make mistakes.

Usage

Run the script by passing the input CSV file and target PNG output path:

Bash



```
python plot_mlfq_timeline.py scheduler_data.csv mlfq_plot.png
```

Expected CSV Format (`scheduler_data.csv`)

Code snippet



```
tick,pid,queue  
0,3,0  
0,4,0  
1,3,1  
5,3,2
```

+ Ask Gemini

Flash Extended ▼

